Collaborative Direct2Consumer: Authentication

João Pedro da Costa Meira

Nº 23505 – Day Regime

Orientation

Patrícia Leite

Academic year 2023/2024

Degree in Computer Systems Engineering

School of Technology

Polytechnic University of Cávado and Ave

Student Identification

João Pedro da Costa Meira

Student Number 23505, day regime

Degree in Computer Systems Engineering

Orientation

Patrícia Isabel Sousa Trindade da Silva Leite

Adjunct Professor at DTCI

Internship Information

Sistrade - Software Consulting, S.A.

Travessa da Prelada 511 Porto 4100-215

Supervision of Inês Caetano and Nuno Rocha

RESUME

Digital authentication (AuthN) is crucial for controlling access and enhancing the user experience. One of the main challenges is the need for repetitive authentication when accessing various services and systems. Therefore, this project aims to solve this problem by developing an authentication module that allows users to be automatically authenticated in additional external modules and services once they have authenticated in the main system.

As part of a larger Textile and Clothing Industry R&D project, this work focuses on the practical issue of multiple authentications for a typical online store user who wants to access services like a virtual fitting room. Thus, initially, a centralized authentication module based on JWT tokens was developed, along with a responsive and user-friendly front-end for the central service platform. In the second phase, two front-end applications with authentication were developed: one to simulate the use of an e-commerce platform and another to simulate an external service. Finally, all modules were integrated, and the authentication system was tested and validated.

On the scope of a company internship, the solution design was aligned with company recommendations, using Node.js and Express.js for the authentication API, and Angular.js for the front-end components. More importantly, the developed solution was successfully validated as part of a full system, demonstrating its ability to provide automated, seamless, and secure access to various resources, thus reducing the pain points of user experience related to repetitive authentication.

ABSTRACT

A autenticação digital (AuthN) é crucial para controlar o acesso e melhorar a experiência do utilizador. Um dos principais desafios é a necessidade de autenticação repetitiva ao aceder a vários serviços e sistemas. Assim, este projeto pretende resolver este problema através do desenvolvimento de um módulo de autenticação que permite que os utilizadores sejam automaticamente autenticados em módulos e serviços externos adicionais, uma vez autenticados no sistema principal.

Como parte de um projeto maior de R&D da Indústria Têxtil e do Vestuário, este trabalho centra-se na questão prática de múltiplas autenticações para um utilizador típico de uma loja online que queira aceder a serviços como um provador virtual. Assim, numa primeira fase, foi desenvolvido um módulo de autenticação centralizado baseado em tokens JWT, juntamente com um front-end reativo e de fácil utilização para a plataforma do serviço central. Na segunda fase, foram desenvolvidas duas aplicações front-end com autenticação: uma para simular a utilização de uma plataforma de comércio eletrónico e outra para simular um serviço externo. Finalmente, todos os módulos foram integrados e o sistema de autenticação foi testado e validado.

No âmbito de um estágio na empresa, o desenho da solução foi alinhado com as recomendações da empresa, utilizando Node.js e Express.js para a API de autenticação, e Angular.js para os componentes de front-end. Mais importante ainda, a solução desenvolvida foi validada com sucesso como parte de um sistema completo, demonstrando a sua capacidade de fornecer acesso automatizado, contínuo e seguro a vários recursos, reduzindo assim os pontos problemáticos da experiência do utilizador relacionados com a autenticação repetitiva.

ACKNOWLEDGMENTS

I would like to express my sincere thanks to everyone who supported and encouraged me during this internship. Your unfailing support has motivated me to persist and continue to grow throughout my years of studies. The last semester presented significant challenges for my personal and academic development. I am especially grateful to all those who have played a crucial role in this journey.

First and foremost, I want to thank my parents and my brother for their unwavering support and continuous encouragement, which have given me strength throughout this experience.

I would also like to thank my supervisors at SISTRADE, Inês Caetano and Nuno Rocha, for their guidance, support and availability, which contributed greatly to the success of my project.

I sincerely thank my supervisor, Patrícia Leite, for her continuous guidance, encouragement and valuable feedback, which were essential to the success of my work.

I would also like to express my gratitude to Sistrade - Software Consulting, S.A. and its employees for providing me with the resources and the opportunity to undertake this challenge.

My friends have also been a great source of support and understanding, and I am deeply grateful for their patience and encouragement throughout this period.

Finally, to everyone who, directly or indirectly, contributed to this project, I sincerely thank you for your effort and support.

Thank you all very much!

INDEX

1. Introduction....1
1.1. Context....2
1.2. Motivation....3
1.3. Objectives....3
1.4. Work Plan....4
1.5. Document Structure....6
2. State of the Art....7
2.1. Security in Computer Systems....7
2.2. Authentication and Authorization concepts....9
2.2.1. Authentication....9
2.2.2. Authorization....10
2.3. Centralized Authentication....12
2.4. Web Frameworks & Libraries....13
2.4.1. Angular.js vs React....13
2.4.2. NodeJS vs Java....15
2.4.3. Microsoft SQL Server vs Oracle Database....17
2.5. Security checking protocols....19
2.5.1. Comparison between protocols....20
2.6. ISO/IEC 27001....21
3. Analysis and Design of the Solution....23
3.1. Technical Requirements....23
3.1.1. Functional Requirements....23
3.1.2. Non-Functional Requirements....24
3.1.3. Assumptions & Constraints....25

3.2. UML Diagrams 25
3.3. Context Diagram 26
3.3.1. System Actors 26
3.3.2. High-level resources 27
3.4. UC Diagrams 27
3.4.1. Authentication UC 28
3.4.2. Application Functionalities UC 30
3.4.3. Application Management UC 32
3.5. Technical System Architecture 34
3.6. Domain Model 35
3.7. Entity Relationship Model 36
3.8. Payment State Diagram 37
3.9. Sequence Diagram 38
4. Mockups 39
4.1. Login 39
4.2. Register 40
4.3. Home Page 41
4.4. Services 42
5. Development of Functionalities 43
5.1. API 43
5.1.1. Configuration and Setup Files 44
5.1.2. AuthN and AuthZ 48
5.1.3. Models 56
5.1.4. Routes 57
5.1.5. Controllers 57
5.1.6. Services 57
5.2. Front-end 58
5.2.1. D2C 58

5.2.2. E-commerce JMI....63
5.2.3. Weather Service....66
5.3. Tests of the Solution....70
5.3.1. Postman....70
5.3.2. Acceptance Tests....72
6. Conclusion....75

INDEX OF FIGURE

Figure 1: Work Plan (Gantt Diagram) 5
Figure 2: Example of CAS implementation (Maratov, 2017) 13
Figure 3: Context Diagram 26
Figure 4: Authentication UC 28
Figure 5: Application Functionalities UC 30
Figure 6: Application Management UC 32
Figure 7: System Architecture 34
Figure 8: Domain Model 35
Figure 9: Entity Relationship Model 36
Figure 10: Payment State Diagram 37
Figure 11: AuthN Sequence Diagram 38
Figure 12: Login Mockup 39
Figure 13: Register Mockup 40
Figure 14: Home Page Mockup 41
Figure 15: Services Mockup 42
Figure 16: API Structure 43
Figure 17: D2C Register 59
Figure 18: D2C Login 60
Figure 19: D2C Home 61
Figure 20: D2C Services Available 62
Figure 21: D2C My Services 63
Figure 22: JMI Store Register 64
Figure 23: JMI Store Login 65
Figure 24: JMI Store Home 66
Figure 25: Weather Service Register 67
Figure 26: Weather Service Login 68
Figure 27: Weather Service Home 69
Figure 28: Register Tests 71
Figure 29: Login Tests 72
Figure 30: AuthN and AuthZ workflow in the D2C Portal system 73

INDEX OF TABLES

Table 1: Comparison of AuthN Methods 10
Table 2: Comparison of AuthZ Methods 11
Table 3: Comparison of Angular and React 14
Table 4: Comparison of NodeJS and Java 16
Table 5: Comparison of MS SQL Server and Oracle Database 18
Table 6: Comparison between security protocols 20
Table 7: Functional Requirements 23
Table 8: Non-Functional Requirements 24
Table 9: Assumptions & Constraints 25
Table 10: Explanation of the Authentication UC 29
Table 11: Explanation of the Application Functionalities UC 31
Table 12: Explanation of the Application Management UC 33

INDEX OF LISTINGS

Listing 1: File .env 45
Listing 2: File app.js 46
Listing 3: File config.js 47
Listing 4: File sequelize.js 48
Listing 5: File server.js 48
Listing 6: const authCookieVerify 49
Listing 7: const checkRoleManager 51
Listing 8: File _authRoutes.js 52
Listing 9: const login 54
Listing 10: const register 56

GLOSSARY

Cater	To satisfy a need or to provide what is wanted or needed by a particular person or group
Eavesdrop	To listen secretly to what is being said in private without the consent of the speaker
Inherent	Existing in something as a permanent, essential, or characteristic attribute, as a natural or basic part of something
Intuitiveness	The quality of being easy and natural to learn, use, or understand
Scalability	It's the ability of a computer process to be used or produced in a range of capabilities
Seamless	Happening without any sudden changes, interruption, or difficulty, the intention is to achieve a seamless transition with continuity of management

ABBREVIATIONS AND ACRONYMS

ABAC	Attribute-Based Access Control
ACL	Access Control Lists
AI	Artificial Intelligence
API	Application Protocol Interface
ARI	Applied Research and Innovation
AuthN	Authentication
AuthZ	Authorization
CAS	Centralized Authentication System
D2C	Direct2Consumer
DAC	Discretionary Access Control
DLP	Data Loss Prevention
DOM	Document Object Model
DTCI	Department of Computing and Information Technologies
FR	Functional Requirements
IAM	Identify Access
IoT	Internet of Things
IPS	Intrusion Prevention System
ISMS	Information Security Management Systems
JSON	JavaScript Object Notation
JWT	JSON Web Tokens
MAC	Mandatory Access Control
MS	Microsoft
NFR	Non-Functional Requirements
NS	Network Security

OIDC	OpenID Connect
ORM	Object Relational Mapper
PBA	Password-based authentication
RBAC	Role-Based Access Control
RDBMS	Relational Database Management System
REST	Representational State Transfer
SAML	Security Assertion Markup Language
SQL	Structured Query Language
SSO	Single Sign-On
TCI	Textile and Clothing Industry
T-SQL	Transact-SQL
UC	Use Case
UML	Unified Modelling Language
MFA	Multi-factor Authentication
HTTP	Hypertext Transfer Protocol
XML	Extensible Markup Language
AC	Assumptions & Constraints
SD	State Diagram

1. Introduction

Integrated into a larger R&D project, the present work focuses on information security, specifically on providing a secure centralized authentication (AuthN) module. The main goal of this project is to create and integrate an external AuthN module into an existing system, which will allow users to be automatically authenticated in additional external modules and services once they have been authenticated in the main system. Therefore, it is intended to provide automated, seamless and secure access to various resources, reducing the usual pain points of user experience related to repetitive AuthN.

Digital AuthN is essential for controlling access and improving user experience. A centralized AuthN system increases security, integrity, and trust in digital ecosystems. The project will be using agile development methodologies to enable fast responses to changes and an iterative and incremental development process.

This essay was written as a part of the Internship Curricular Unit, which took place between February 22 and June 13, 2024, as a requirement for the Degree in Computer Systems Engineering at the School of Technology of the Polytechnic University of Cávado and Ave[1]. The internship was hosted by Sistrade – Software Consulting, S.A.[2], a company based in Porto. Founded in the year 2000, SISTRADE is an international company that provides software development and consultancy services for various sectors of activity, such as industry and services.

SISTRADE's team of employees is highly specialized in web-based software development and has a vast knowledge of different business processes. They work in a healthy and fun environment, allowing everyone to overcome obstacles and face new challenges. The internship was conducted under a hybrid program.

[1] https://est.ipca.pt/
[2] https://sistrade.com/

1.1. Context

In digital the modern digital environment, the user experience is fundamental to the success of online platforms. A common problem faced by users when accessing different services on the same platform is the repeated need to log in. This recurring process can result in user dissatisfaction and an inconsistent experience.

SISTRADE's research project, insert on Textile and Clothing Industry (TCI), aims to solve this problem by developing a centralised authentication service. Through this service, users will be able to authenticate once and seamlessly access various services from different TCI companies integrated into a centralized platform. For clarification of reader, it is important to define the following concepts:

- E-commerce platforms – online stores in which users can buy TCI articles (e.g., Zara[3])
- D2C Services – Set of services available in the D2C Platform (e.g., recommendation engine, virtual fitting room)
- D2C Platform – Main platform responsible for providing different services that can be integrated into e-commerce platforms. The main users of this platform are the managers of e-commerce platforms, which can buy services to be used by their customers when visiting the online store.

Following the previous nomenclature, as a baseline example, if Zara clients want to virtually try an outfit, they will have to authenticate twice: once on the Zara website, and again to access the virtual fitting room. To simplify this process, we are offering a new authentication module that will allow users to log in just once (i.e., in Zara website), and have access to all the services provided on the platform. This single login will grant automatic access to the virtual fitting room without the need for a second authentication. Currently, requiring a second login for external services when accessing Zara's virtual fitting room adds unnecessary complexity and detracts from the user experience. By centralising authentication, we aim to simplify user interactions, reduce redundancy and improve overall usability.

[3] https://www.zara.com/pt/

1.2. Motivation

From the beginning, this project was something I was deeply motivated to do during my internship, as it was perfectly aligned with my objectives. Creating an Application Protocol Interface (API) has always been a primary goal, and this project presented an ideal opportunity to achieve it. The use of widely used technologies such as Node.js and Express.js gave me the confidence that I would adapt well and prosper in the API architecture.

Although my experience with Angular.js was limited, I was excited by the prospect of learning and working with this technology. The opportunity to expand my skill set and contribute to front-end development using Angular.js was an advantage for me.

In addition, the internship's emphasis on developing a centralized authentication module was particularly attractive. Understanding and implementing a centralized authentication module is crucial for future projects, as it not only simplifies interactions with users but also increases security across platforms. This knowledge is invaluable and will certainly benefit the future professional.

1.3. Objectives

The main aim of the current work is to develop a centralized authentication module to simplify access to a variety of services from a single platform. After an in-depth analysis of security concepts in information systems, with special emphasis on authentication and authorization, the following objectives were defined:

O1: Develop a centralized Authentication module based on JWT tokens to streamline the authentication process for user identities.

O2: Design a responsive and user-friendly front-end for the D2C Platform using Angular.js.

O3: Develop a front-end application with authentication to simulate the usage of an e-commerce platform.

O4: Develop a front-end application with authentication to simulate an external service, such as a weather forecast.

O5: Integrate the AuthN module into the systems outlined in O2, O3 and O4.

O6: Validate the AuthN module as part of the full system.

1.4. Work Plan

This project was developed using an agile methodology, with the work plan outlined in a Gantt Diagram as shown in Figure 1. The "*Relatório de Estágio*" stage was carried out throughout the entire duration of the internship and involved the preparation of this report.

The initial stage, "*Estado de Arte*", focused on investigating computer system security, AuthN and authorization (AuthZ) concepts, centralized AuthN systems and protocols, and comparing available solutions.

Then it analysed functional and non-functional requirements, defined the solution's architecture, and created mockups for the application in the "*Análise e desenho da solução*" stage.

An optimal stage, "*Melhoria pessoal dos conceitos das Frameworks*", allowed to improve the knowledge on certain days. The final Stage "*Implementação e teste da solução*", crossed the entire development period and included creating the database, developing the API, validating functionality, designing the solution's front-end, documenting software development, and deploying to the company's server.

Versão 1.16 feito na data 12/06/2024		Duração Prevista	Fevereiro						Março				Abril				Maio					Junho				Estado de execução	% realização de tarefas
Atividade/Tarefa	**Descrição sumária da Tarefa**		S5	S6	S7	S7	S8	S9	S10	S11	S12	S13	S14	S15	S16	S17	S18	S19	S20	S21	S22	S23	S24	S25	S26		
Atividade 1	**Relatório de Estágio**	**16 semanas**																								Completed	100%
Atividade 2	**Estado de Arte**	**5 semanas**																								Completed	100%
Tarefa 2.1.	Segurança nos sistemas informáticos	1 dia																								Completed	100%
Tarefa 2.2.	Conceitos de Autenticação e Autorização	2 dias																								Completed	100%
Tarefa 2.3.	Diferentes tipos de Autenticação e Autorização	2 dias																								Completed	100%
Tarefa 2.4.	Sistemas e Protocolos de Autenticação Centralizados existentes	2 dias																								Completed	100%
Tarefa 2.5.	Pesquisa e comparação de diferentes soluções disponiveis	3 dias																								Completed	100%
Tarefa 2.6.	Análise das vantagens e desvantagens das frameworks	1 dia																								Completed	100%
Atividade 3	**Análise e desenho da solução**	**6 semanas**																								Completed	100%
Tarefa 3.1.	Definição dos casos de uso	4 dias																								Completed	100%
Tarefa 3.2.	Análise de requisitos (funcionais e não funcionais)	3 dias																								Completed	100%
Tarefa 3.3.	Definição da arquitetura da solução	3 dia																								Completed	100%
Tarefa 3.4.	Realização das Mockups para a Aplicação	1 dia																								Completed	100%
Atividade 4 (opcional)	***Melhoria pessoal dos conceitos das Frameworks***	**6 semanas**																								Completed	100%
Tarefa 4.1.	*Melhorar os conceitos em Frontend (Angular)*	2 dias																								Completed	100%
Tarefa 4.2.	*Revisão dos Conceitos de Backend (NodeJS & Express)*	2 dias																								Completed	100%
Atividade 5	**Implementação e Teste da Solução**	**11 semanas**																								Completed	100%
Tarefa 5.1.	Criação da Base de Dados para a Solução	2 dias																								Completed	100%
Tarefa 5.2.	Desenvolvimento da API da aplicação	6 semanas																								Completed	100%
Tarefa 5.3.	Validação Funcional	2 semanas																								Completed	100%
Tarefa 5.4.	Realização do frontend	3 semanas																								Completed	100%
Tarefa 5.5.	Criação da documentação do Software	1/2 semanas																								Completed	100%

Figure 1: Work Plan (Gantt Diagram)

1.5. Document Structure

This document consists of six main chapters, together with an introduction, each focusing on specific aspects of the project to develop a centralized authentication module.

Introduction: This section provides an overview of the aims of the project and contextualises it, outlining the motivation, objectives and work plan.

State of the Art: This chapter analyses the problem, discussing fundamental concepts of security in computer systems, such as AuthN, AuthZ, centralised authentication systems and security protocols. It also includes comparisons between web frameworks and libraries, such as Angular vs React and NodeJS vs Java, as well as database servers such as Microsoft SQL Server vs Oracle Database.

Analysis and Design of the Solution: This chapter systematically surveys the system's functional and non-functional requirements and describes Unified Modelling Language (UML) diagrams such as context diagrams, use cases, class diagrams and entity-relationship diagrams. It also presents the system's domain model and technical architecture.

Mockups: It shows visual representations of the user interfaces for the application, created using wireframes. The mockups offer a clear guide for the development process, displaying the login, registration, home page, and services page designs.

Development of Functionalities: This chapter describes the technologies chosen to implement the project and justifies the choices. It details the structure of the API developed in Node.js and Express.js, the frontend in Angular.js, and presents the configuration and installation files. The implemented functionalities, including AuthN and AuthZ, are also described. It also analyses the results obtained and describes the tests carried out on the system, evaluating performance, security and usability.

Conclusion: This section summarises the main achievements of the project and ensures that the objectives have been met. It also discusses lessons learnt, difficulties encountered and suggestions for future work.

Each chapter is essential for a global understanding of the project, from the initial conception to the final evaluation of the results.

2. State of the Art

Aiming to provide a brief contextualization regarding state-of-the-art technologies used in the field, this chapter presents the frameworks that will be used throughout the internship. It includes an explanation of computer security, AuthN, centralized AuthN systems, and available solutions.

2.1. Security in Computer Systems

These days the internet has become part of our lives, and it has transformed the way we live positively. As the technology increases, it also brings along various computer security threats with it. Computer security, also known as cybersecurity, is defined as the protection of computer systems, and data from unauthorized access, attack, or theft, ensuring the confidentiality, integrity, and availability of information. It is composed of multiple layers of protection spread across the computers, networks, programs, or data that one intends to keep safe.

Nowadays, there are several different types of computer security solutions, which directly impact an organization's physical and digital infrastructure (Check Point Software Technologies, 2024). These types include Network Security (NS), Cloud Security, Endpoint Security, Application Security, and Internet of Things (IoT) Security designed to identify and block informatic attacks.

Within the NS, it is possible to find data and access approaches such as Data Loss Prevention (DLP), Identify Access (IAM), or Next-generation Firewall (NGFW) application controls to enforce safe web use policies. NS also relies on advanced and multi-layered network threat prevention technologies such as Intrusion Prevention System (IPS). These measures allow authorized users to access network resources while blocking malicious actors from carrying out exploits and threats. NS safeguards the integrity of network infrastructure, resources, and traffic to prevent these attacks and minimize their financial and operational impact (IBM, 2024).

With the increased adoption of cloud computing by Information Technology (IT) organizations, Cloud Security has become a major priority. Cloud Security strategies include cybersecurity solutions, controls, and services, such as IAM which enables policy enforcement for users accessing on-premises and cloud services, DLP ensures the security of regulated cloud data, and Security Information and Event Management (SIEM) uses Artificial intelligence (AI) technologies to automate threat monitoring, detection, and response by correlating log data across various platform and digital assets (IBM, 2024).

From another perspective, Endpoint Security is the cybersecurity approach to defending endpoints, such as desktops, and mobile devices from potential threats. The Endpoint Protection Platform (EPP) is a solution used to detect and prevent various types of security threats, such as malware attacks, and provide investigation and remediation capabilities that allow for prompt responses to incidents and alerts.

Complementing the Endpoint Security, the Application Security is a comprehensive approach that aims to protect data or code within an application. It encompasses the security considerations that happen during application development and design, as well as systems and approaches to safeguard apps after deployment. Next-generation Antivirus (NGAV) closes this gap by utilizing more advanced endpoint protection technologies, such as AI and Machine Learning (ML), to identify new malware. It examines various elements, including URLs and IP addresses, to detect and prevent threats (Aarness, 2024).

Aligned with Industry 4.0 expansion, the adherence to IoT devices has significantly boosted organization productivity. However, it also increased the exposition of organizations to new cyber threats. Indeed, threat actors exploit vulnerable devices for malicious purposes, such as gaining access to corporate networks or another bot in a global bot network. IoT Security measures consist of several steps, including Device Discovery and Classification to manage devices effectively, Auto-segmentation to enhance NS, performance, and resource management, and IPS as a virtual patch to prevent exploits. In some cases, small agents can be added to the firmware of the device to prevent runtime attacks and exploits (Check Point Software Technologies, 2024).

2.2. Authentication and Authorization concepts

AuthN and AuthZ are two critical information security processes that administrators use to protect systems and information. Authentication validates the identity of a user or service, while Authorization determines the actions a user can take after access. Although they are different, AuthN and AuthZ share similarities, as both are integral parts of the access control process and depend on identity verification. Notably, as it is not possible to authorize a user or service before identifying it, AuthN always comes before AuthZ (Identity, 2024).

2.2.1. Authentication

The main purpose of AuthN is to confirm an individual or organization's identity. This is a crucial process that enables access to applications or data. Users must enter their username and password, which are then checked against the system's database records. If the credentials match, access is granted, indicating the user's legitimacy.

In such a digital era, the applicability of AuthN has become crucial and highly diverse. For instance, AuthN can use various methods to verify a user's identity, including known physical devices, One-time Pins (OTP) sent by Short Message Service (SMS) or app, and biometric authentication mechanisms such as fingerprints. These methods ensure that users are authenticated and can access systems, guaranteeing their security (Frontegg, 2022).

When it comes to AuthN and security, there are several other options, with strengths, weaknesses, and appropriate use cases:

- Password-based Authentication (PBA) is commonly used, but vulnerable to attacks such as force attacks, phishing, and password guessing.
- Token-based Authentication uses small pieces of data generated by an authentication server and passed to the client, which can be session or refresh tokens, such as JSON Web Tokens (JWT)[4] and Open Authorization (OAuth) Tokens[5].
- Single Sign-On (SSO) lets users authenticate once and access many apps without re-entering credentials, improving user experience and security.

[4] https://jwt.io/
[5] https://www.oauth.com/oauth2-servers/access-tokens/

- Multi-factor Authentication (MFA) requires multiple forms of evidence to verify identity and provides better security than PBA.

Table 1 presents a comparison between the authentication methods:

Table 1: Comparison of AuthN Methods

Feature	*PBA*	*JWT*	*SSO*	*MFA*
Authentication Type	Password-based	Token-based	Centralized	Multi-factor
Primary Function	Based on user-entered passwords	Access token, API authentication	Centralized authentication, improved user experience	Additional security layer
Relation to Password Authentication	Dependent on user-generated passwords	Reduces reliance on passwords	Rely on initial password-based authentication	Implemented alongside password-based authentication
Security Considerations	Ensuring password complexity, hashing, and secure storage	Prevent token tampering, Cross-site Scripting (XSS), and Cross-site Request Forgery (CSRF) attacks	Trust in third-party identity provider	Balance additional factors with user convenience
Implementation Complexity	Moderate to High	Moderate to High	High	Moderate
Example	Traditional username or email and password login	Header, Payload and Signature Separated by dots xxxxx.yyyyy.zzzzz	Implementing Google's SSO in its products	Facial recognition or Fingerprints

2.2.2. Authorization

AuthZ determines the level of user or service access in a system. The system will grant individual permissions based on their verified identities, which specifies the actions that users can perform after access. AuthZ systems range widely across technological environments.

In line with AuthZ principles, Access Control Lists (ACLs) regulate which users or services are permitted to enter a given digital space. They implement rules of allowance or denial that are determined by the user's level of AuthZ. If a standard user wishes to make alterations that could impact the system's safety, the ACL may disallow access. On the other hand, administrators have the necessary AuthZ to make changes that affect security, so the ACL will allow them to perform that action (Identity, 2024). Examples of distinct ACLs are public data accessible to everyone, internal data for employees, and confidential data restricted to a selected group.

In AuthZ and security measures, it's crucial to recognize the list of methods available, each one offering distinct strengths and weaknesses:

- Role-based Access Control (RBAC) assigns permissions based on users' roles in an organization. It simplifies administration and ensures access to only necessary resources.
- Attribute-Based Access Control (ABAC) evaluates user, resource, and environment attributes to determine access permissions. This provides refined control over access policies.
- Mandatory Access Control (MAC) limits resource access based on information sensitivity and user AuthZ.
- Discretionary Access Control (DAC) provides resource access and offers more flexibility than MAC, but it requires meticulous management.

Table 2 presents a comparison between AuthZ methods:

Table 2: Comparison of AuthZ Methods

Feature	*RBAC*	*ABAC*	*MAC*	*DAC*
Access Control Model	Role-based	Attribute-based	Mandatory	Discretionary
Primary Function	Access control based on job roles	Access control based on attributes	Enforce security policies	Access control based on the owner's discretion
Access Decision	User's role and associated permissions	Based on attributes and policies	Determined by system	Owner's role, permissions, and policies
Security	Lower risk exposure	Fine-grained access control	High	Higher risk exposure
Implementation Complexity	Moderate	High	Moderate to High	Low

2.3. Centralized Authentication

Centralized authentication is a method where users' credentials are verified by a single server. This approach is different from SSO, where users are authenticated only once and can access multiple services or servers without being prompted for their credentials again. With centralized authentication, users are still required to enter their credentials for each service they access, but they only need to remember one set of credentials for all services that use the same centralized authentication system.

Centralized Authentication System (CAS) was designed to be used as an authentication measure. Users can authenticate the untrusted applications with that, which does not reveal the password to these applications as the entire authentication is performed on the CAS Servers. In an SSO, the users only need to authenticate once to access all the servers and services they need, the network automatically remembers the user's information and uses it each time a new resource has to be accessed, and that's the centralized authentication process. So, with that, CAS can provide an SSO solution for multiple web applications to provide a more seamless end-user experience, while the CAS server is a trusted source that can be used for authentication purposes.

To do this, the CAS protocol and the authentication flow involve several parties such as the client's web browser, which is software incorporated into web applications that use the CAS service. Web application is the application seeking authentication, while the back-end service is where the CAS protocol can involve a database server that doesn't have its own Hypertext Transfer Protocol (HTTP) interface but can communicate with a web application.

Using CAS protocols has many benefits for use, which include convenience, consistency, and less effort needed for applications and security, we can take a little extra time to set up, but it can provide an end-user experience that has less friction. It can also be easier to maintain over time, as there is a centralized server to deal with on the system instead of managing authentication processes within each web application (Okta, 2023).

The CAS authentication process involves four key steps to authenticate users. First, the user requests access to a service, which redirects them to the CAS server. Next, the server generates a unique ticket and a CAS SSO cookie and then forwards the ticket to the application server. Finally, the ticket is validated by the server and access

is granted if successful. This process is critical for ensuring secure access to applications and services across multiple platforms, as shown in Figure 2.

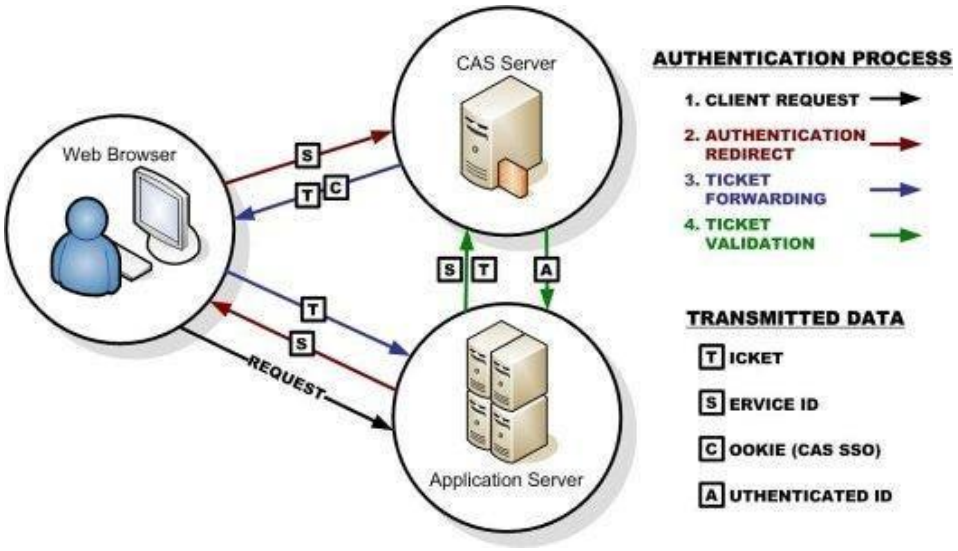


Figure 2: Example of CAS implementation (Maratov, 2017)

2.4. Web Frameworks & Libraries

A Web Framework are pre-built software tools that allow developers to create web applications without having to start from scratch. This saves time and reduces the risk of errors in the code. Developers can choose a single web framework or combine multiple frameworks depending on the project requirements (Technologies, 2019).

Web libraries are collections of modular functions, classes, and methods that offer specific, focused functionalities for web development. Unlike web frameworks, libraries are not comprehensive structures but provide individual tools or components.

2.4.1. Angular.js vs React

Web developers frequently rely on either Angular.js[6] or React[7] while creating robust and dynamic web applications. Although these two JavaScript[8] frameworks share some similarities, they differ significantly, giving them their unique characteristics. Before delving into their technical aspects, let's briefly compare these frameworks.

[6] https://angularjs.org/
[7] https://react.dev/
[8] https://developer.mozilla.org/docs/Web/JavaScript/

Angular, an open-source framework maintained by Google[9], is specifically designed to create front-end web applications. It addresses the challenges encountered when developing single-page, cross-platform, and performant applications. It uses a component-based architecture to create applications and it's built using TypeScript[10] (DataFlair, 2024).

On the other hand, React is an extremely versatile and efficient JavaScript library that caters to several front-end development requirements. Developers can conveniently create user interface (UI) components using a component-based architecture and easily manage views. With their philosophy, developers can build interactive and complex UIs, and develop fast and scalable applications for all platforms (Dhaduk, 2024). Before diving into the technical details, let's compare the two frameworks in question with Table 3.

Table 3: Comparison of Angular and React

Feature	*Angular*	*React*
Type	Full-fledged structural framework	JavaScript-based library
Language	TypeScript	JavaScript (JSX script)
Purpose	Develop dynamic web applications	Build interactive UI components
Developed and maintained by	Google	Meta and community
App structure	Fixed and complex platform, component-based framework	Flexible, component-based
Data binding	Two-way	One-way
Advantages	Strongly typed, object-oriented and large community	Virtual DOM for efficient rendering and flexibility to learn
Disadvantages	Steep learning curve and Complex syntax	Lack of documentation and no built-in support for features

[9] https://www.google.pt/
[10] https://www.typescriptlang.org/

The decision to embrace Angular as the framework for this project was based on its widespread usage within SISTRADE. This choice enables seamless collaboration among the team and Angular's robust component model and dependency injection for creating dynamic web applications. The extensive functionalities and capabilities of Angular further solidify its suitability for this project.

2.4.2. NodeJS vs Java

In the realm of web development, both Node.js[11] and Java[12] are highly prevalent programming languages. In contrast, Node.js is renowned for its rapidity, non-blocking capabilities, and event-driven methodology, while Java is celebrated for its resilience, durability, and expandability. To make a knowledgeable choice between two languages, aspects such as performance, scalability, and community assistance must be weighed. By comprehending the unique attributes of each language, developers can determine the optimal solution for their requirements.

Node.js is an open-source and cross-platform runtime environment and library that is used for running web applications outside the client's browser. It is used for server-side programming, and primarily deployed for non-blocking, event-driven servers, such as traditional websites and back-end API services (EPAM Systems Inc., 2024).

Java is a powerful programming language and computing platform that was introduced by Sun Microsystems in 1995. Over the years, it has evolved into a reliable platform that powers a significant portion of the digital world. Java is widely used for developing innovative products, digital services, and applications that are designed to meet future needs (Oracle J. &., 2024).

[11] https://nodejs.org/pt/
[12] https://www.java.com/

Both are popular programming languages that are often used for web development, while they have some similarities, there are also some important differences as shown in Table 4.

Table 4: Comparison of NodeJS and Java

Features	*NodeJS*	*Java*
Type	JavaScript-based runtime environment	Object-oriented programming language
Language	JavaScript	Java
Purpose	Server-side scripting, network applications, and back-end development	Enterprise applications, web applications, and mobile applications
Developed and maintained by	Ryan Dahl, Joyent, and the community	Sun Microsystems, Oracle
Front-end development	Yes, with libraries like Express.js and Socket.io	No, it's used just for back-end development
Performance	Non-blocking, event-driven, and fast	Robust, stable, and scalable
Scalability	Highly scalable with horizontal scaling	Highly scalable with vertical scaling
Concurrency	Single-threaded, non-blocking I/O model	Multi-threaded, blocking I/O model
Advantages	JavaScript-based, non-blocking and large community	Object-oriented and good for enterprise applications
Disadvantages	Single-threaded, limited for CPU-bound tasks and limited built-in features	Detailed syntax, steep curve for beginners and requires more code to do the same task

The selected framework for the project will be React, as it’s the framework currently utilized by SISTRADE in the Applied Research and Innovation (ARI) Department.

2.4.3. Microsoft SQL Server vs Oracle Database

Both Microsoft[13] (MS) and Oracle's Structured Query Language[14] (SQL) servers serve the same purpose, which is the Relational Database Management System (RDBMS), an RDBMS allows users to create and maintain relational databases.

While MS SQL Server is an RDBMS developed by MS. It supports American National Standards Institute[15] (ANSI) SQL, which is the standard SQL language, however, it uses its implementation and that is Transact Structured Query Language[16] (T-SQL) (Microsoft, 2024).

Oracle Database is a reliable and powerful RDBMS developed by Oracle Corporation. It supports SQL and PL/SQL and is known for its robustness, stability, and scalability, it manages various data types and provides advanced security and high availability features such as encryption, authentication, Real Application Clusters (RAC), and Data Guard. It's widely used across various industries to manage large volumes of data efficiently and securely (Oracle, 2024).

[13] https://learn.microsoft.com/en-us/sql/ssms/
[14] https://www.oracle.com/database/sqldeveloper/
[15] https://www.ansi.org/
[16] https://learn.microsoft.com/training/modules/get-started-transact-sql-programming/

These two databases are very popular database management systems, each with its unique strengths and features. While they share some similarities, there are also some significant differences to consider when choosing the right one as shown in Table 5.

Table 5: Comparison of MS SQL Server and Oracle Database

Feature	*MS SQL Server*	*Oracle Database*
Type	Relational database management system	Relational database management system
Language	Enterprise data management, business intelligence, and analytics	Enterprise data management, business intelligence, and analytics
Purpose	T-SQL	Procedural Language (PL) or SQL
Developed and maintained by	Microsoft	Oracle Corporation
Performance	High performance, optimized for Windows platforms	High performance, optimized for various platforms
Cloud Integration	Seamless integration with Microsoft Azure	Seamless integration with Oracle Cloud
Advantages	Good for Windows environments and good for enterprise data management	A wide range of features and capabilities, also good for enterprise data management
Disadvantages	Complex to configure and manage, limited support for open-source technologies	Limited support for some open-source technologies and is less user-friendly than others

The database that will be utilized is MS SQL Server, as it’s the preferred database for the ARI department.

2.5. Security checking protocols

Security checking protocols, also known as NS Protocols, are network protocols that ensure communication between two or more parties. These protocols provide various security services, such as authentication, confidentiality, and integrity, to protect the communication from unauthorized access, and eavesdropping.

JWT is an open standard used to share information between two parties securely, a client and a server. In most cases, it's an encoded JSON containing a set of declarations and a signature, it is usually used in the context of other authentication mechanisms, such as OpenID Connect (OIDC), to share user-related information. JWTs can be encrypted but are usually encoded and signed (Okta, 2023).

SSO is a technology that combines several different application login screens into one, with that a user must only enter their login credentials one time on a single page to access all their Software as a Service (SaaS) applications (Cloudflare, 2024).

Security Assertion Markup Language (SAML) is an open-source protocol and SSO standard. Its primary role in online security is that enable access to multiple web applications using one set of login credentials. It works by passing authentication information through signed Extensible Markup Language (XML) documents between an identity provider (IdP) and a web application (One Identity, 2024).

OIDC is an identified authentication protocol that is an extension of Oauth to standardize the process for authenticating and authorizing users when they sign in to access digital services. It provides authentication, which means verifying that users are who they say they are (Okta, Inc., 2024).

2.5.1. Comparison between protocols

The following Table 6 provides a comparison of four protocols, they are the JWT, SSO, SAML, and OIDC. Each protocol has its strengths and weaknesses, which will be discussed in further detail.

Table 6: Comparison between security protocols

Features	*JWT*	*SSO*	*SAML*	*OIDC*
Token type	Token-based	Authentication protocol	XML-based	Authentication protocol
Specification	IETF RFC 7519	Oauth 2.0	OASIS	OpenID Connect Core 1.0
Technologies	JSON	Oauth 2.0, Rest	SOAP, XML	Oauth 2.0, REST, JSON
Token Format	JSON	JSON	XML	JSON
Token Storage	Client-side	Client-side	Server-side	Server-side
Token Size	Small	Small	Medium	Medium

The student has chosen JWT because it is the protocol that is more comfortable for him to use in this application, which he has used before in some projects that have done at the university.

2.6. ISO/IEC 27001

The ISO/IEC (acronym) standard, which carries the number 27001, holds a widely accepted status as the universal benchmark for information security management systems (ISMS). It lays down the fundamental requirements that an ISMS should fulfil, offering guidance to companies of all sizes and sectors on how to create, execute, sustain, and continuously improve a system for managing information security.

Conforming to the ISO/IEC 27001 standard[17] indicates that a business or organization has put in place a comprehensive framework for managing risks relating to the security of their data. This framework adheres to the established best practices and principles of the International Standard (ISO/IEC, 2022).

[17] https://www.iso.org/standard/27001

3. Analysis and Design of the Solution

The aim of this chapter it's to show an analysis of the solution, such as the functional requirements (FR) and non-functional requirements (NFR), realization of use cases (UC), and more UML diagrams, and the definition of the architecture.

3.1. Technical Requirements

3.1.1. Functional Requirements

FR are the features and requirements that the system will have to meet the UCs as shown in Table 7.

Table 7: Functional Requirements

FR	*Name*	*Description*
FR01	Personal Account	The app should allow users to create accounts and log in using credentials such as email or through an external integration.
FR02	Edit Profile	Users should be able to change their profile information whenever they want.
FR03	Change Password	User can change their passwords if the account is not linked to a third party.
FR04	Payments	Users need to make payments for the services they have purchased to access them.
FR05	User Role Management	The app should allow administrators to assign and change user roles to ensure appropriate access and permissions are granted.
FR06	AuthN and AuthZ	The app should use JWT for user authentication and ensure that all requests to protected resources are authorized.

3.1.2. Non-Functional Requirements

NFRs are characteristics that the system must respect so that each of the system's FRs is delivered and serves the user in the best way possible, as you can see below in Table 8.

Table 8: Non-Functional Requirements

NFR	*Name*	*Description*
NFR01	System Security	The app should have a strong authentication service, requiring users to provide strong passwords.
NFR02	User Permissions	User roles must be well defined and enforced, so each type of user only has access to their specific pages.
NFR03	Maintenance	The System must be easy to maintain and update, avoiding any inconveniences or procedures that could lead to excessive delays in repair.
NFR04	System Requirements	The application should run on Windows 10 and must be able to run on Windows 11 without any changes or problems.
NFR05	Intuitiveness	How simple the application it's to understand the interface for the user easily.
NFR06	Data Encryption	The sensitive data as passwords will be encrypted to prevent unauthorized access.

3.1.3. Assumptions & Constraints

All assumptions and constraints (AC) shown in Table 9 that have a relevant impact on the project must be explicitly mentioned and documented.

Table 9: Assumptions & Constraints

AC	*Name*	*Description*
AC01	Deadlines	The project must be ready for implementation by 07 June 2024 at the latest.
AC02	Languages	The application will only support English at the beginning, if its time left, the app can also be in Portuguese
AC03	D2C Usage	The D2C platform is intended exclusively for managers. Only managers have access to D2C, while regular users have access to e-commerce platforms and services.
AC04	E-commerce Platform	JMI store is an e-commerce platform available to users
AC05	Service Accessibility	Weather service is a service available to users
AC06	Functionality	Both the JMI store and Weather service have their functionalities

3.2. UML Diagrams

Starting with UML diagrams, which include use cases, class diagrams, and others, it's first necessary to know what UML diagrams are, it's a software modelling language that has a vocabulary and rules for combining terms and allows for the construction of conceptual and physical representations of a system.

It was developed by 3 researchers, Booch, Jacobson, and Rumbaugh, and resulted from the combination of the methods/techniques of each one involved, Booch used class, state, and object diagrams, while Rumbaugh used the Object Modelling Technique (OMT) technique and Jacobson used Object Oriented Software Engineering (OOSE).

It supports various activities in the process such as analysis which facilitates communication and visualization, design through the creation of models, and coding in a way that generates automatically based on class and interaction models. It can also serve as the foundation of the system's documentation, as it's a globally accepted software modelling language (Dennis, 2009).

3.3. Context Diagram

A context diagram provides a comprehensive and conclusive overview of a system's interactions with external entities. It highlights external actors and data flow, with the system placed at the core. It is an effective tool for defining system boundaries, facilitating communication, and serving as a foundation for creating more intricate data flow diagrams.

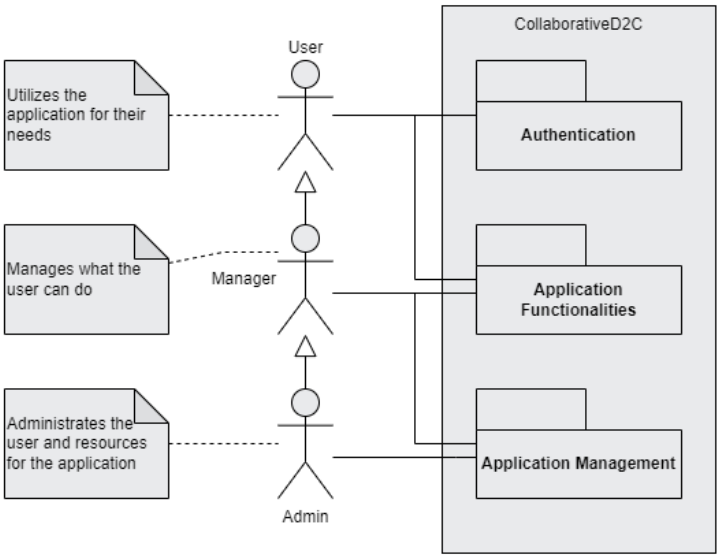


Figure 3: Context Diagram

3.3.1. System Actors

The system shown in Figure 3 will consist of three actors, which are described below:

- **User –** This represents the regular user of the system who has access to the services.
- **Manager –** The user can subscribe to the services every month and can also view other services and companies that offer the same services.

- **Admin –** This represents the system administrator who has access to all the resources. The admin manages the users and ensures the security of the application.

For easy understanding, let's make the following comparison: the **user** corresponds to the regular customer who uses the e-commerce platform (a typical customer who makes purchases on the Zara platform and has a personal account with their transaction history), the **Manager** represents the Zara owner, who purchases services (such as fitting room access) to be provided to Zara clients when using the website. Finally, the admin is the individual responsible for managing and overseeing the platform.

3.3.2. High-level resources

The system presented in Figure 3 will consist of three distinct resources, each serving a specific purpose:

- **Authentication –** This resource contains the various methods available for users to verify their identity and access the application.
- **Application Functionalities –** This resource provides regular users with access to a range of functions, including checking purchased services and browsing partner companies and their offerings
- **Application Management –** This resource enables admins to perform a range of functions, such as managing users, controlling services and companies, and ensuring the smooth overall operation of the application.

3.4. UC Diagrams

A UC diagram represents the actions performed by the users, it's a functionality of the system seen by the users and a type of high-level interaction between the actors and the system. In addition, the UC is a logical model that describes actions without specifying how they are implemented, it describes the actions of the system and the actions of the actor interacting with it and is identified by its inherent use of purpose (Unhelkar, 2020).

3.4.1. Authentication UC

The below UC in Figure 4 describes the steps for a user to register on a service provider's platform. Users want to successfully sign up for the service, while the service provider wants to grant access upon completion. Optional functionalities, such as Two-Factor Authentication (2FA) for greater security or SSO for a simpler registration process, can also be included in the process.

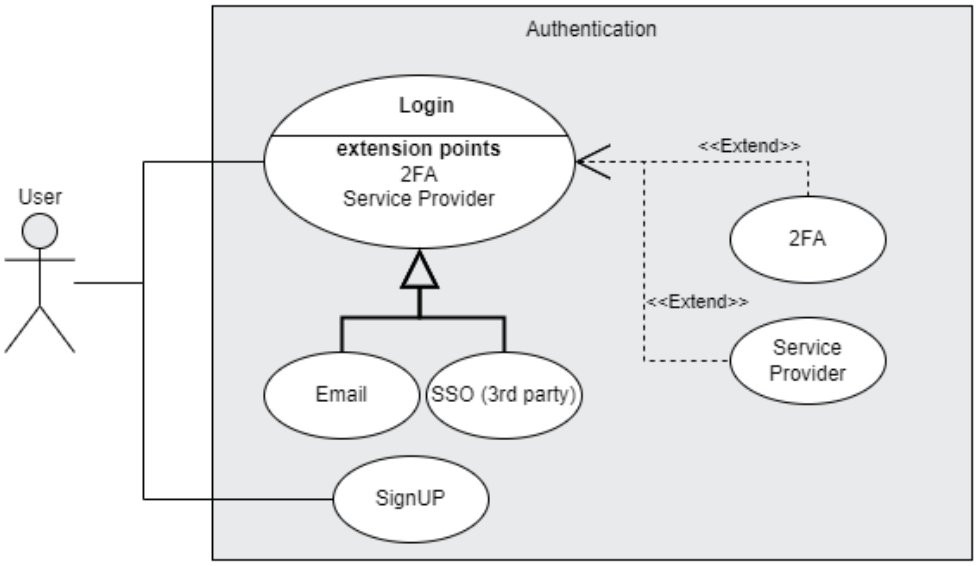


Figure 4: Authentication UC

The following Table 10 provides a detailed explanation of the Authentication UC.

Table 10: Explanation of the Authentication UC

UC name: *Authentication*

Actor	User
Brief Description	The system allows users to login to the application using their registered credentials with various available methods
Precondition	The user must be registered in the system and have valid credentials (email and password)
Description	1. The user selects the login option in the application 2. The system prompts the user for their credentials 3. The user enters their registered credentials 4. The system validates the entered credentials against the stored user data
Alternative scenario	5. If the user enters invalid credentials, the system will display an error message and prompt the user to re-enter their credentials (steps 2 and 3) 6. If the account is locked, the system will inform the user and provide instructions on unlocking it, such as initiating a password reset
Postcondition	After successful validation, the user can access the application

3.4.2. Application Functionalities UC

The UC in Figure 5 shows how registered users engage with the application's different features. It details the process users follow to access functions, including navigating menus and using specific tools. The system also considers potential obstacles, such as inadequate permissions or technical issues.

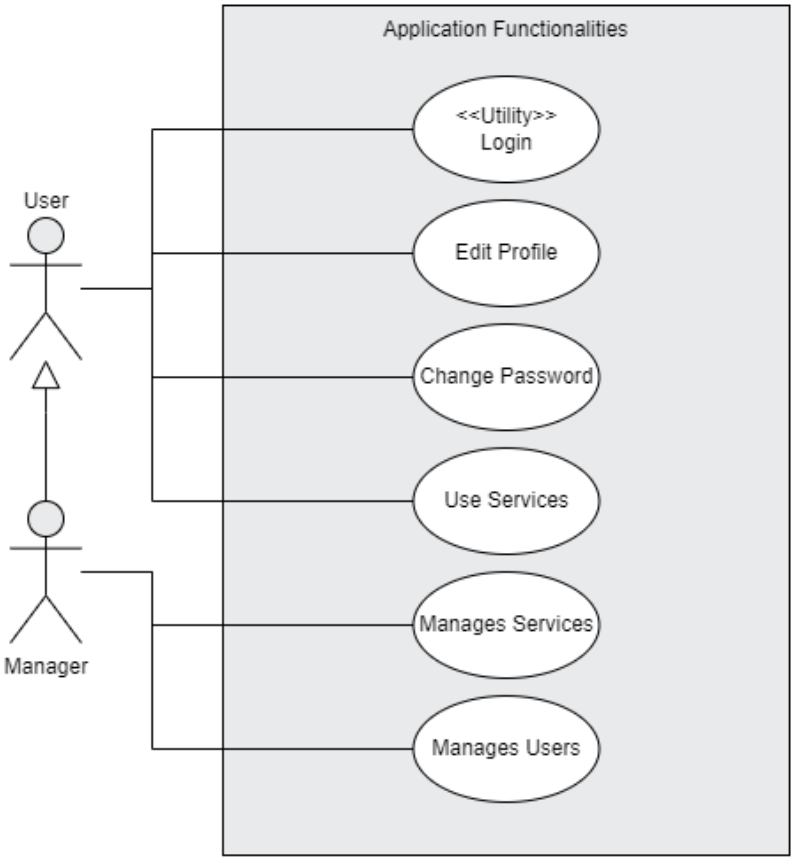


Figure 5: Application Functionalities UC

The following Table 11 provides a detailed explanation of the Application Functionalities of UC.

Table 11: Explanation of the Application Functionalities UC

UC name: *Application Functionalities*

Actor	User and Manager
Brief Description	The system allows registered users to access and utilize the various functionalities offered by the application
Precondition	The user must be successfully logged in to the application
Description	1. Upon successful login, the user is presented with the main interface of the application 2. After choosing the desired functionality, the system displays the chosen option and provides the user with the necessary options and interactions 5. Users can interact with the services as intended, such as searching for a service or modifying their profile 6. The system processes the user's actions and provides relevant feedback, and updated account details 7. The manager can manage the users and the services
Alternative scenario	3. The user may encounter some functionalities that are inaccessible due to insufficient permissions 4. The system also displays appropriate error messages when technical issues appear.
Postcondition	The user successfully uses the selected feature and accomplishes the desired result within the application. The system remains accessible for the user to explore and engage with other features. While the manager can control the services and the users.

3.4.3. Application Management UC

The system shown in Figure 6 demonstrates the user registration process for a system overseen by an administrator. The administrator can create, view, and analyse user statistics, as well as manage services, and companies, and suspend users.

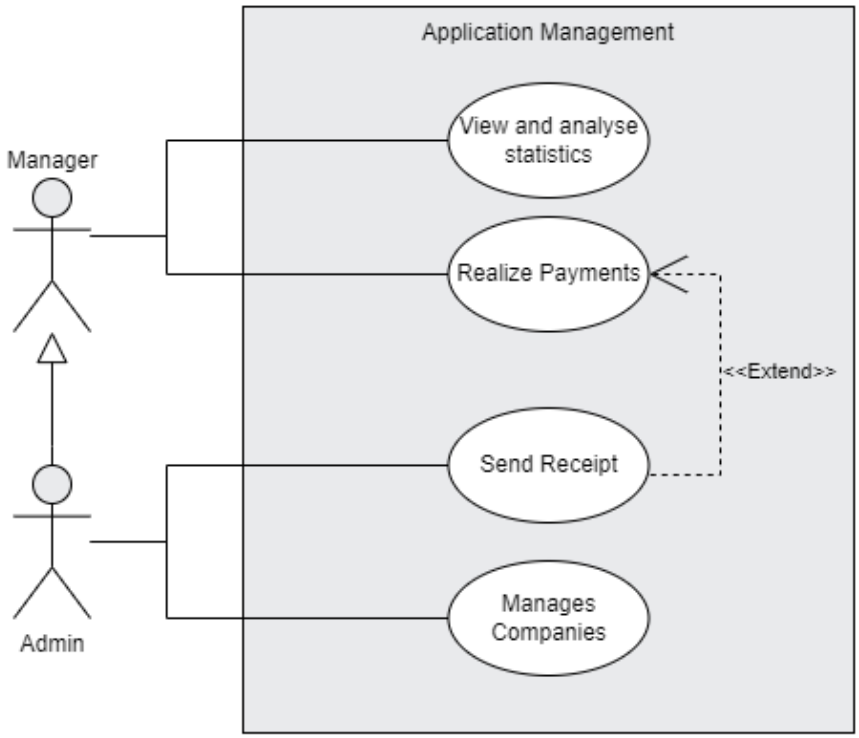


Figure 6: Application Management UC

The following Table 12 provides a detailed explanation of the Application Management UC.

Table 12: Explanation of the Application Management UC

UC name: *Application Management*

Actor	Manager and Admin
Brief Description	Manage the application's features and improve the user experience
Precondition	The administrator has the required permission to oversee the application
Description	1. The administrator has full control over the application's functionality and user interface through the management control 2. The manager can oversee the application performance and user activity to pinpoint areas for enhancement and guarantee a seamless user experience 3. The administrator sets up security settings and access controls to protect the application and user data, allowing authorized access for all
Alternative scenario	5. The system prevents unauthorized access to specific features 6. The system also presents error messages and may restrict functionality during technical difficulties
Postcondition	The administrator successfully implements changes and upholds the application's optimal performance and user experience

3.5. Technical System Architecture

The systems architecture represented in Figure 7 consists of a web application that comprises both front-end and back-end components. The front end is created using AngularJS and is responsible for the user interface. Users interact with the front-end through a web browser, which communicates with the front-end server over the internet.

The Representational State Transfer (REST) API is created using Node.js and Express.js. It serves as an intermediary between the front-end and back-end and manages requests from the front-end. It also communicates with the database server.

While MS SQL Server is used for the database due to its robustness and scalability. The database server is connected to the REST API server through the API.

The system's architecture uses widely adopted technologies, which results in a modern and efficient system that simplifies data management and provides a user-friendly interface.

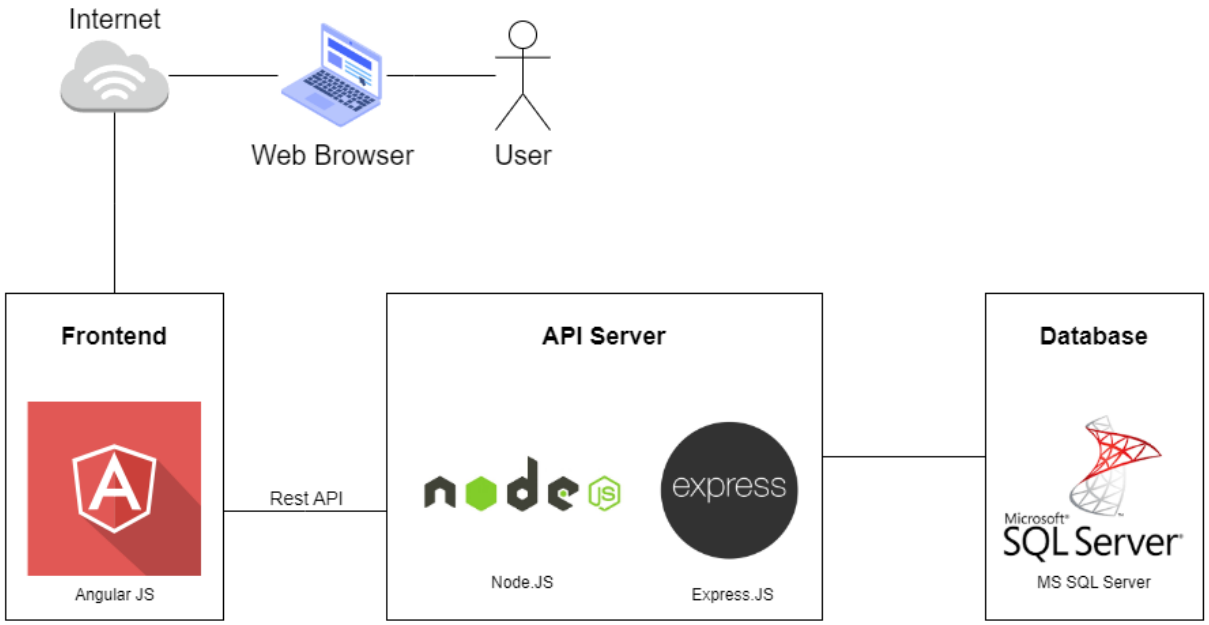


Figure 7: System Architecture

3.6. Domain Model

The domain model is a conceptual model of the domain that incorporates both behaviour and data. As shown in Figure 8 the system allows the primary user to access different activities, corresponding to the application's pages. To ensure smooth navigation within the company's website, the user's ID is saved, which eliminates the need to log in constantly.

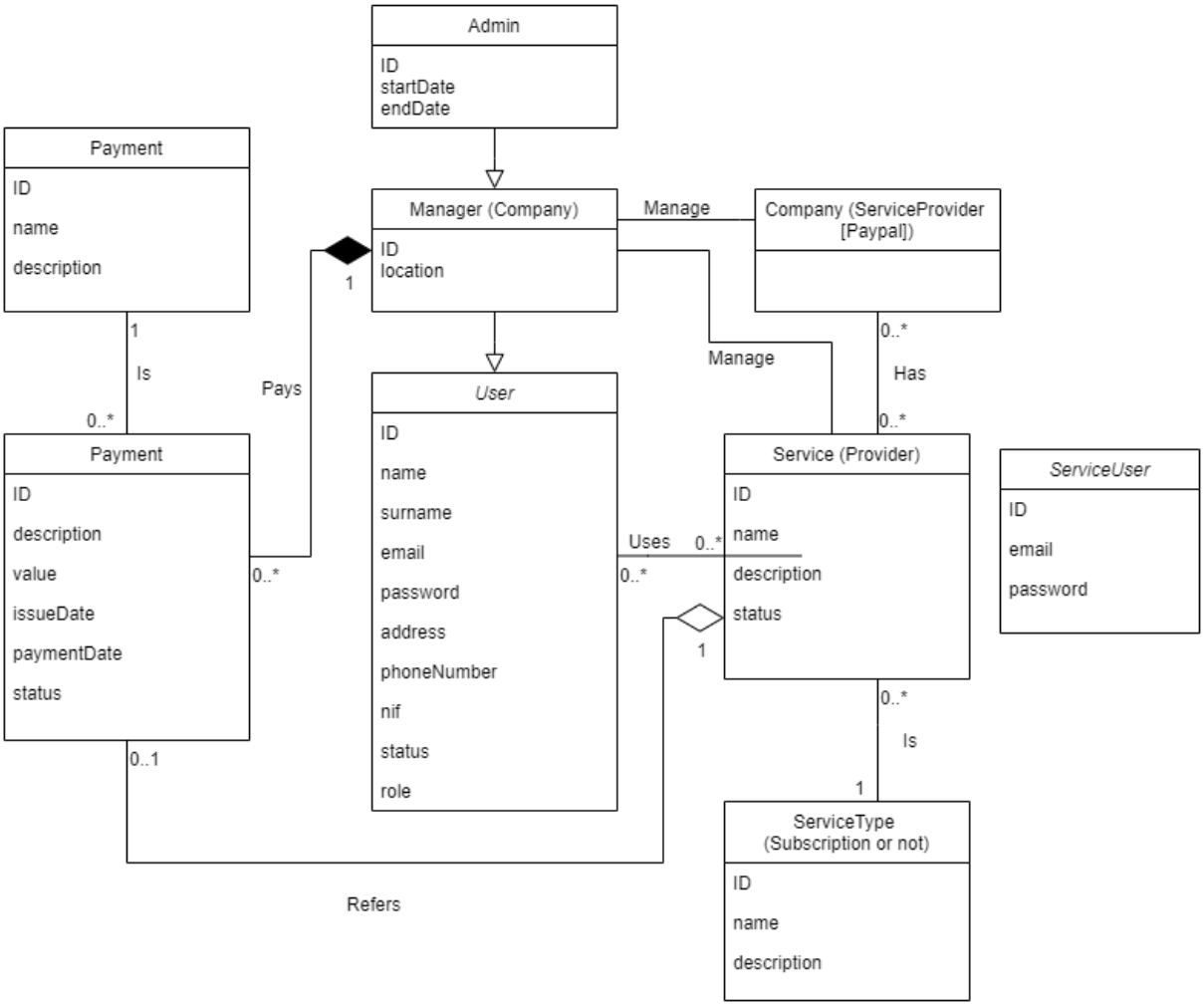


Figure 8: Domain Model

3.7. Entity Relationship Model

The Entity Relationship (ER) model, like Figure 9 is a conceptual data model utilized to symbolize entities and their corresponding relationships in a database. This model is instrumental in visualizing and comprehending the database's structure, which can be highly beneficial In the process of database design and management.

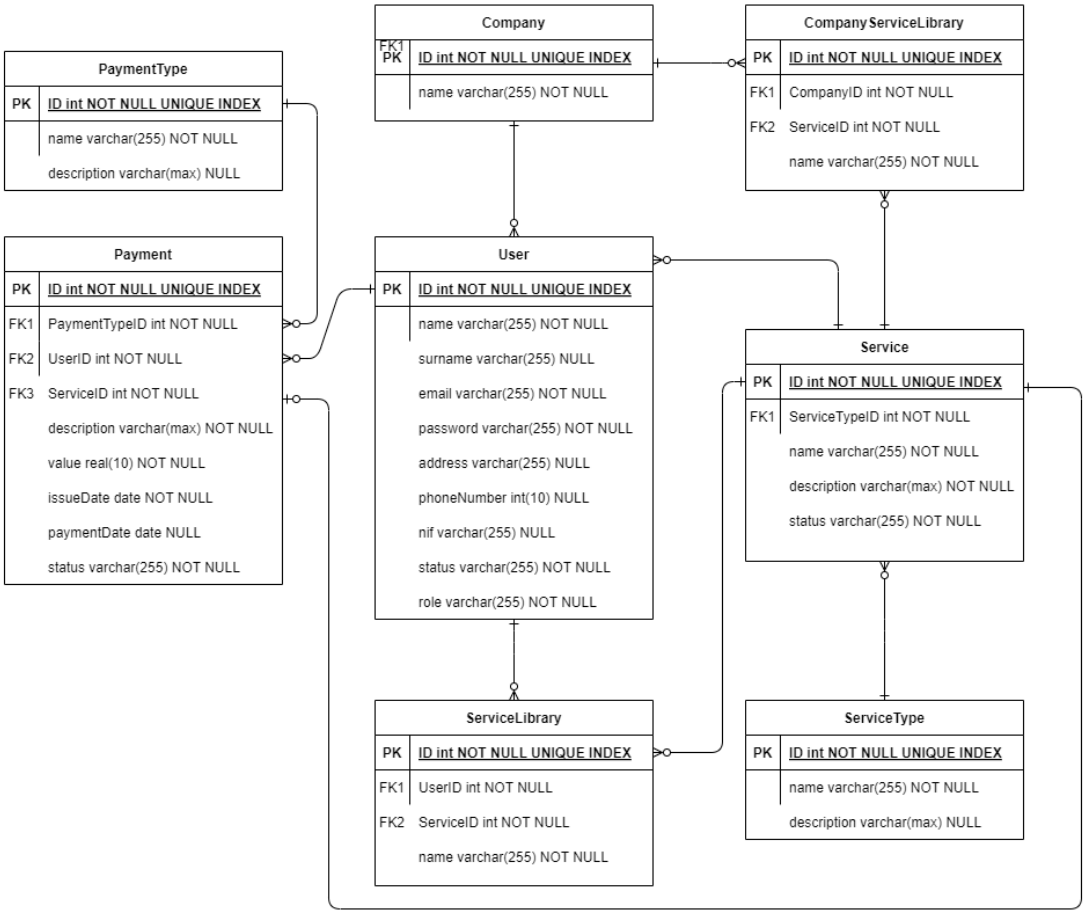


Figure 9: Entity Relationship Model

3.8. Payment State Diagram

A state diagram (SD) serves as a visual representation that shows the numerous states in which a system can exist, as well as the transitions between these states.

The SD in Figure 10 shows the different stages a payment goes through from initiation to completion. The process begins when the user makes a payment, which is indicated by the “Made the payment” state. The system then submits the payment information and enters a “Pending” state, during which it awaits confirmation from the payment gateway or bank. If confirmation is received, the state changes to “Paid”, indicating a successful transaction.

If confirmation is not received within a specified timeframe, the payment times out, and the state transitions to “Payment deadline expired”. The user can resolve the late payment by paying the overdue amount, which will transition the state back to “Paid”. Once a payment is successful, the process ends.

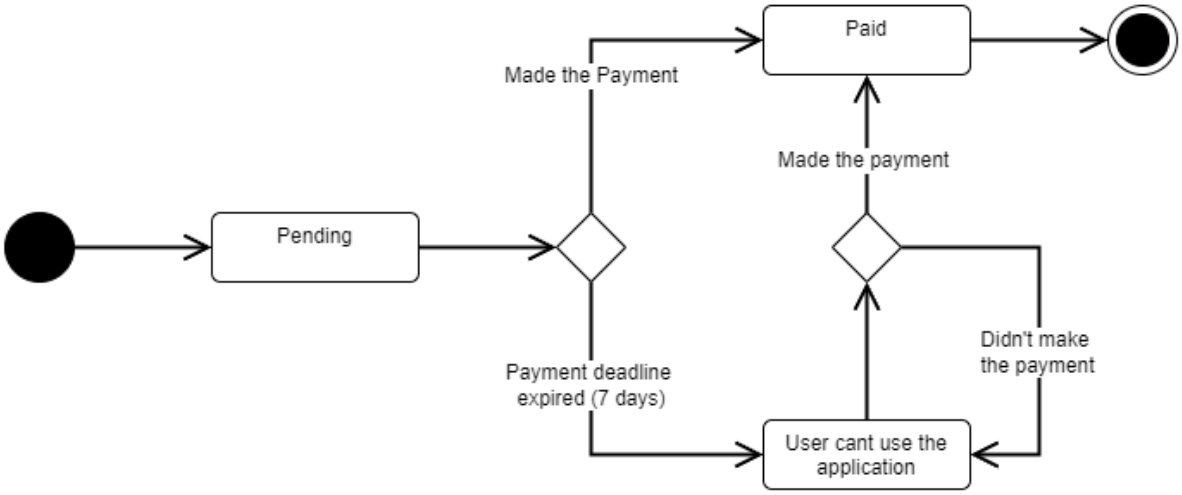


Figure 10: Payment State Diagram

3.9. Sequence Diagram

The sequence diagram shown below in Figure 11 shows the interaction between the system components during the registration and login processes. For registration, a user sends a POST request to the /register endpoint, handled by the **register()** function in the authentication controller. The controller calls the user service, which adds the user's data to the database via the user model. A message is then sent to the user.

During login, the user sends a POST request to the /login endpoint, which is handled by the **login()** function in the authentication controller. The controller calls the user service to authenticate the user's credentials using the user model. If successful, a message is sent to the user.

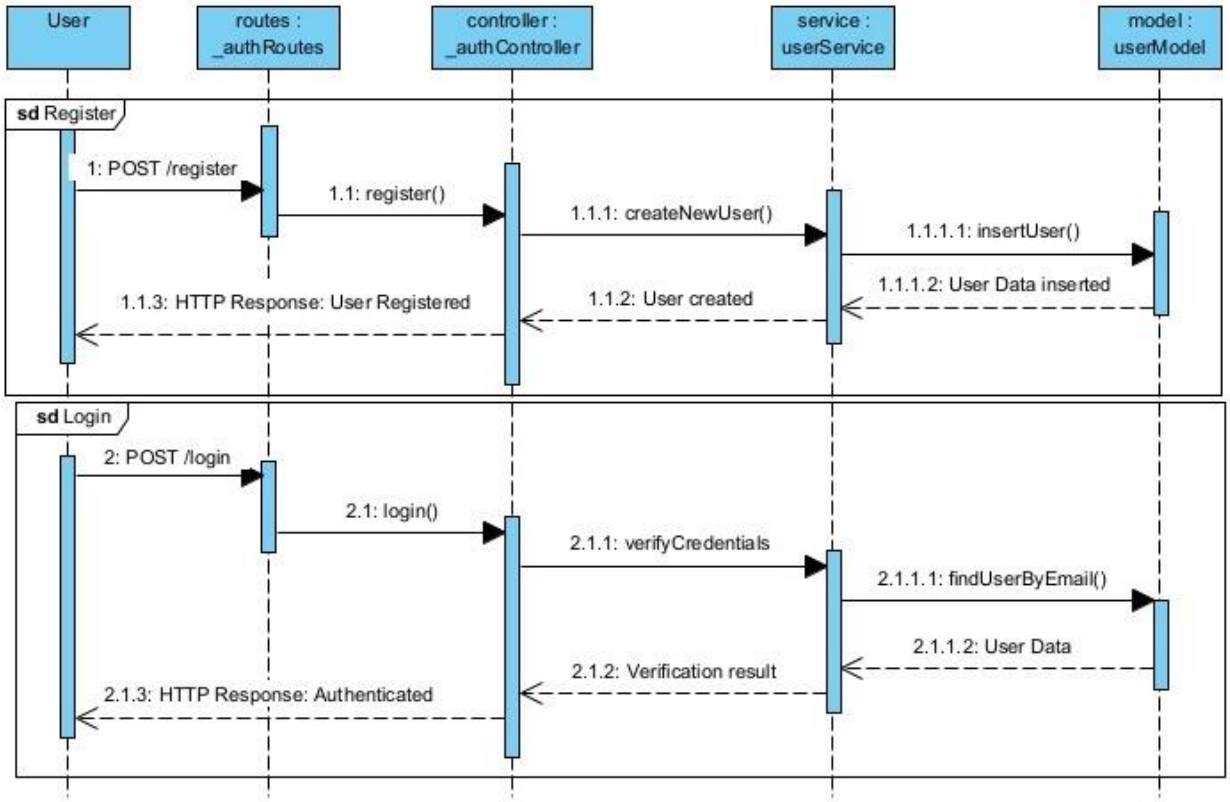


Figure 11: AuthN Sequence Diagram

4. Mockups

Mockups are essential visual tools that play a crucial role in transforming design concepts into visual representations. These representations provide a clear and precise guide for the development process. The student opted to use Balsamiq wireframes[18] to create the following mockups due to their familiarity with this tool. It's important to note that these mock-ups represent an initial stage and are distinct from the outcome, given that they were created midway through the project.

4.1. Login

Figure 12 shows the mock-up of the login screen for the application. It features input fields for "E-mail" and "Password" with accompanying icons. The "Log in" button is distinct with its white colour. Additionally, there are links for password recovery and creating a new account, both displayed in blue at the bottom of the screen.



Figure 12: Login Mockup

[18] https://balsamiq.com/wireframes/

4.2. Register

Figure 13 shows a mock-up of the registration screen for the application, which features input fields for "Name", "Surname", "Email" and "Password", accompanied by relevant icons. Users also have the option of registering using a Google account. The prominent "Register" button provides easy access and a blue link at the bottom directs existing users to the login page.



Figure 13: Register Mockup

4.3. Home Page

This mock-up for the application, shown in Figure 14, features a prominent logo in the top section. Below the logo is a navigation menu with links to "Home", "Services", "Companies", "About us" and "Contact". The main part of the page includes the title, positioned in the centre of the screen. Below the title, there is space for a paragraph of text accompanied by icons and inline elements providing detailed information or descriptions. The footer includes information on copyright and icons for the social networks Instagram, Twitter, Facebook and LinkedIn. In addition, there is an icon in the top right-hand corner indicating the user's profile options.

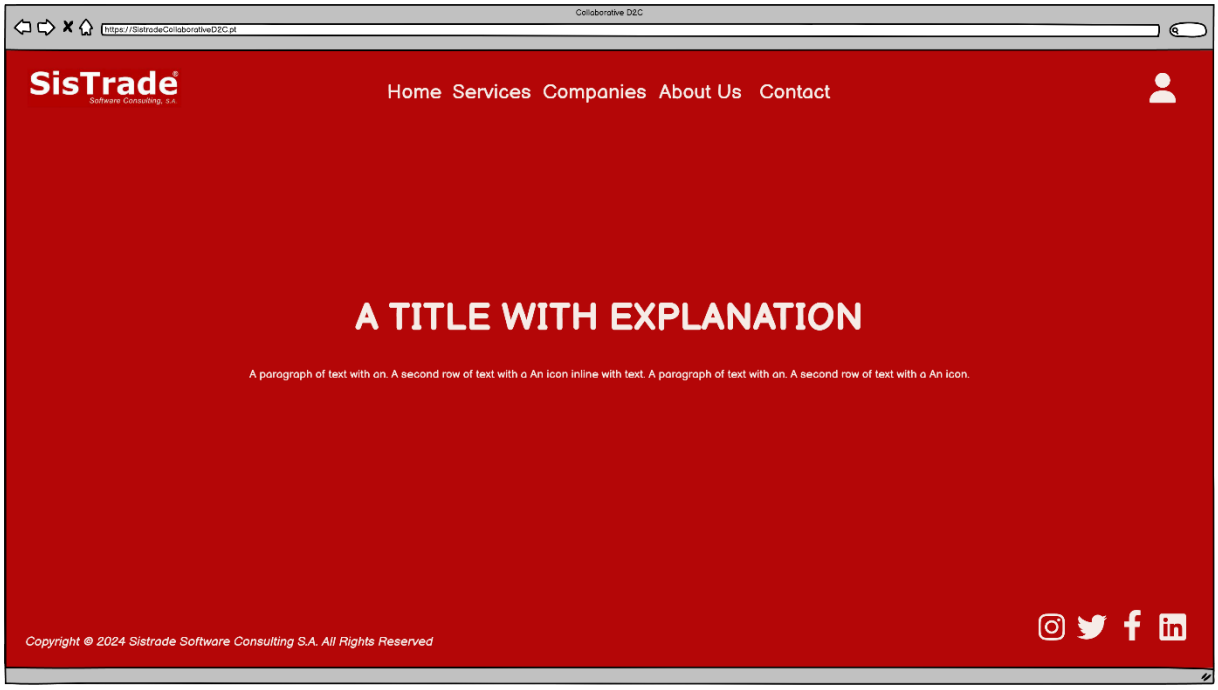


Figure 14: Home Page Mockup

4.4. Services

Figure 15 shows the mock-up of the "Services" page of the application. At the top of the page is the logo and a navigation menu with links to "Home", "Services", "Clients", "About Us" and "Contact". The main section is entitled "Services" and is centred at the top. Below the title, you'll see placeholders for various service offerings, presented in a grid layout. Each service is presented by a square image placeholder above a bold title. The grid layout consists of two rows, with four services per row, ensuring a clear and orderly presentation. Finally, in the top right-hand corner of the page, there is an icon for the user's profile options.

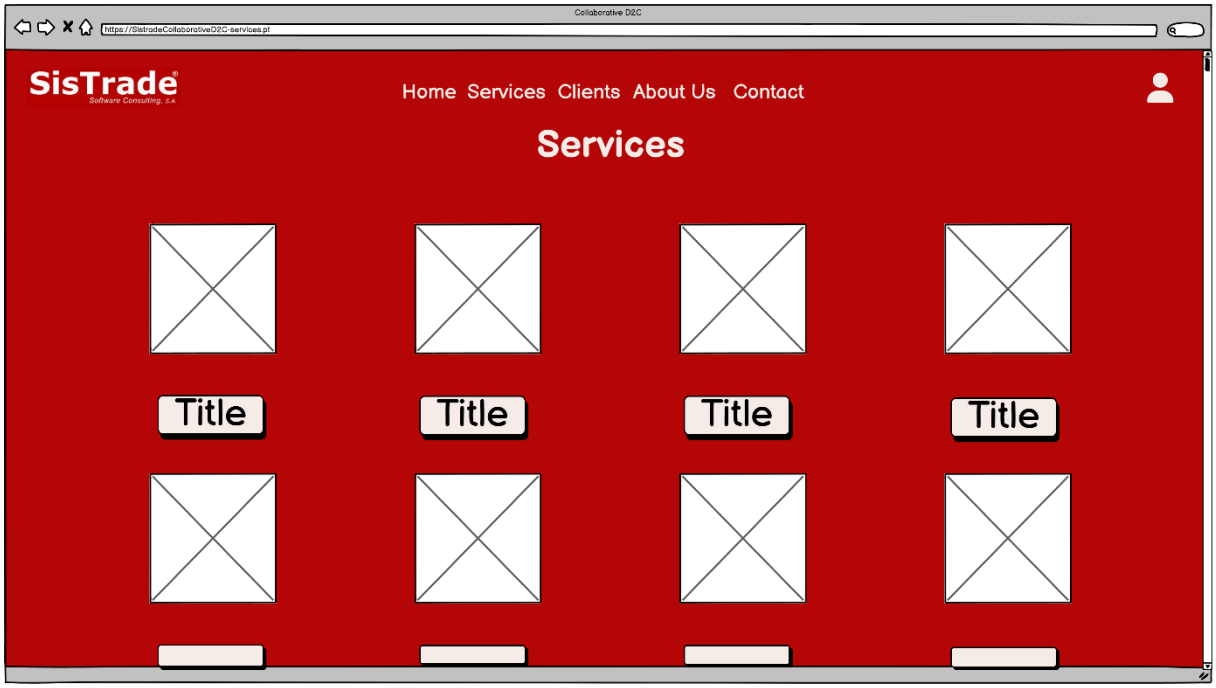


Figure 15: Services Mockup

5. Development of Functionalities

This chapter outlines the development of the proposed features, organized into several sub-sections. These sub-sections contain the system's AuthN and AuthZ process, the API, the front-end, and the solution's testing procedures.

5.1. API

As detailed in the project report, the API was created using Node.js and Express.js, both widely used technologies for building high-performing and scalable applications.

The API structure is organized into folders and files, as depicted in Figure 16, each with specific responsibilities:

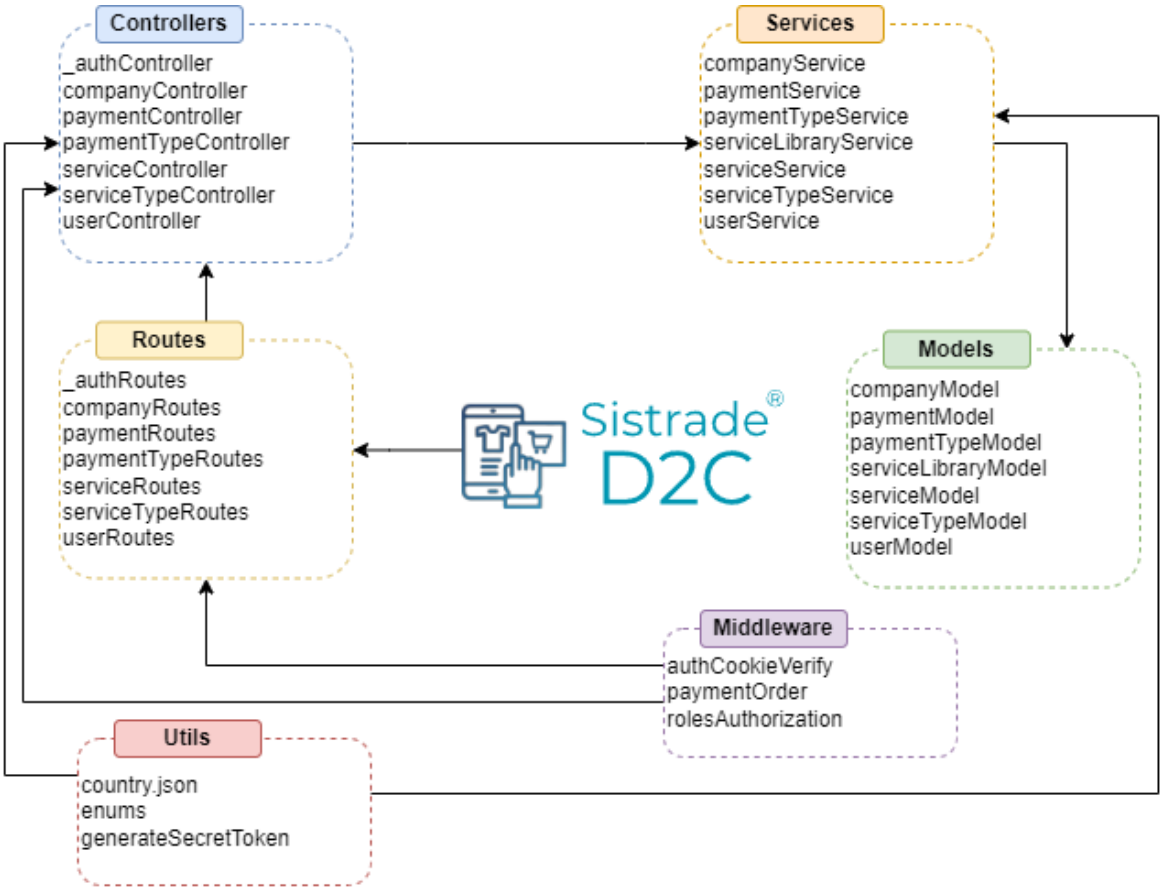


Figure 16: API Structure

Controllers contain the business logic and HTTP request handlers.

Middleware Includes intermediary functions that process requests before they reach the controllers.

Models define the database schemas and models using an Object Relational Mapper (ORM) like Sequelize.

Routes map the API's endpoints, associating each route with a specific controller.

Services contain services that encapsulate reusable business logic.

SpecsOpenAPI stores the API documentation following the Open API standard to facilitate communication and integration with other teams and systems.

Tests include automated tests to ensure the quality and correct functioning of the API.

Utils contain utility functions used in various parts of the application.

5.1.1. Configuration and Setup Files

This section provides a detailed explanation of the main configuration and installation files that form the spine of our Node.js and Express.js API. These essential files are fundamental for initialising the application, managing settings, and establishing a connection to the database. A full understanding of these files will lay a solid foundation for understanding the structure and functionality of the API. The files covered will include the **.env** file, which stores environment variables that configure the application. It uses a **generateSecretToken.js** utility to securely manage sensitive information. The **app.js** file initialises the Express application, defines the middleware and configures the routes. It serves as the entry point to the application's logic. The file **config.js** contains configuration settings for the application, such as database connection details and API settings. The **sequelize.js** configures Sequelize ORM, establishing the connection to the database and loading models. It's crucial for interactions with the database. Finally, the server.js file starts the server, listens for incoming requests, and links everything together when importing and using the Express application.

5.1.1.1. .env

The **.env** file as shown in Listing 1 is essential for storing environment variables that configure the application. These variables are loaded into the application using the dotenv package, ensuring that critical information like database credentials and API keys are not hard coded in the application code. This .env file leverages a utility function from **generateSecretToken.js** to securely manage the secret Token.

As the information in the .env file is sensitive, we can't display its actual contents here. However, we can provide an example using fictional data to explain how it's structured.

```
NODE_ENV=development

#express serve config
PORT=3000
HOST=localhost
HOST_URL=http://localhost:3000

#sql server config
SQL_USER=JOAOMEIRA
SQL_PASSWORD=*****
SQL_DATABASE=ESTAGIO
SQL_SERVER=xxx.xxx.xxx.xxx
SQL_ENCRYPT=false

SECRET_TOKEN=2af6490c436c4eb94fa36ecf2aa7882828e048f2190d747c9bac79f2
a787dee01b3a9c769fa5a6d5593183820d8acdf350137d0b8eff3ebb2498ebfbdc2db4ad
```

Listing 1: File .env

5.1.1.2. app.js

This file **app.js** shown below in Listing 2 acts as the entry point of the Express application. It initializes middleware to handle JSON and URL-encoded payloads, parses cookies and configures routes. Additionally, it includes error-handling middleware to manage errors during the request-response cycle.

```
      'use strict'
      const express = require('express');
      const cookieParser = require('cookie-parser');
      const cors = require('cors');
      const bodyParser = require('body-parser');
      const app = express();

      app.use(cors());
      app.use(bodyParser.json());
      app.use(express.urlencoded({ extended: true }));
      app.use(cookieParser());

      const baseUrl='/collaborativeD2C/api'
      app.use(baseUrl, require('./routes/_authRoutes').routes);
      app.use(baseUrl, require('./routes/_authServiceUserRoutes').routes);
      app.use(baseUrl, require('./routes/companyRoutes').routes);
      //app.use(baseUrl,
require('./routes/companyServicelibraryRoutes').routes);
      app.use(baseUrl, require('./routes/paymentRoutes').routes);
      app.use(baseUrl, require('./routes/paymentTypeRoutes').routes);
      app.use(baseUrl, require('./routes/serviceLibraryRoutes').routes);
      app.use(baseUrl, require('./routes/serviceRoutes').routes);
      app.use(baseUrl, require('./routes/serviceTypeRoutes').routes);
      app.use(baseUrl, require('./routes/userRoutes').routes);

      module.exports = app;
```

Listing 2: File app.js

5.1.1.3. config.js

The **config.js** shown in Listing 3, centralizes the configuration settings for the application, including database connection details and the secret token used for JWT authentication. This segregation of configuration data enhances maintainability and makes it easier to manage different environments.

```
      'use strict'
      const dotenv = require('dotenv');
      const assert = require('assert');

      dotenv.config();

      const {PORT, HOST, HOST_URL, SQL_USER, SQL_PASSWORD, SQL_DATABASE,
SQL_SERVER} = process.env;

      const sqlEncrypt = process.env.ENCRYPT === "true";

      assert(PORT, 'PORT is required');
      assert(HOST, 'HOST is required');

      module.exports = {
          port: PORT,
          host: HOST,
          url: HOST_URL,
          sql: {
              server: SQL_SERVER,
              database: SQL_DATABASE,
              user: SQL_USER,
              password: SQL_PASSWORD,
              options: {
                  encrypt: sqlEncrypt
              }
          }
      }
```

Listing 3: File config.js

5.1.1.4. sequelize.js

The **sequelize.js** file in Listing 4, sets up Sequelize, the ORM used for database interactions. It establishes the database connection using the configuration details provided in config.js and loads all the models defined in the application.

```
      const { Sequelize } = require('sequelize');
      const config = require('./config');

      const sequelize = new Sequelize(config.sql.database, config.sql.user,
config.sql.password, {
          host: config.sql.server,
          dialect: 'mssql',
          dialectOptions: {
              encrypt: config.sql.options.encrypt,
          },
      });

      module.exports = sequelize;
```

Listing 4: File sequelize.js

5.1.1.5. server.js

Finally, the **server.js** shown below in Listing 5, initiates the server and listens for incoming requests. It imports the Express app from **app.js** and specifies the port on which the server will run. These files effectively tie together the setup and configuration, launching the application.

```
      const app = require("./app");
      const config = require("./config");

      app.listen(config.port, () => {
          console.log('Server listening on http://localhost:' +
config.port);
      });
```

Listing 5: File server.js

5.1.2. AuthN and AuthZ

Having seen the configuration files, a closer look into the implementation of AuthN and AuthZ in our Node.js and Express.js API. AuthN is responsible for verifying the identity of users, while AuthZ determines their access levels and permissions within the system. Both are crucial for ensuring the application's security and proper access control. This chapter will cover the main components and their interactions.

The **authCookieVerify.js** file is responsible for checking the JWT in cookies for user AuthN. The middleware **rolesAuthorization.js** enforces RBAC endpoints, while the **_authController.js** file contains controllers for login and registration features.

These components collaborate to provide a secure and efficient AuthN and AuthZ system, ensuring that only authenticated users can access protected resources and that users have the appropriate permissions to perform actions in the application.

5.1.2.1. authCookieVerify.js

The **authCookieVerify.js**, as shown in Listing 6, contains middleware that verifies the presence and validity of a JWT in the request headers or cookies. If a valid token is found, it decrypts the token and adds the user information to the request object, allowing for further processing by subsequent middleware or route handlers.

```
      const authCookieVerify = async (req, res, next) => {
          let token = req.headers.token;


      if (!token) {
               token = req.cookies?.token;
               if (!token) {
                   return res.status(401).json({ message: 'Token not
provided' });
               }
          }


          try{
               const user = jwt.verify(token, process.env.SECRET_TOKEN);
               req.user = {user};
               next();
          }
          catch (error){
               // res.clearCookie("token");
               return res.status(401).json({ error: `${error.message}` });
          }
      }
```

Listing 6: const authCookieVerify

5.1.2.2. rolesAuthorization.js

The **rolesAuthorization.js** file contains middleware functions responsible for implementing RBAC ensuring that users can only access resources and perform actions permitted by the roles assigned to them in the system. This file includes other functions that check whether the authenticated user has the necessary role to access specific routes.

The **checkRoleManager** function shown in Listing 7, serves as a middleware to verify that the user making the request holds the “Manager” role. To begin, the function retrieves the JWT from the AuthZ header of the incoming request. Should the header be missing or fail to contain a token, the function returns a 401 Unauthorized status along with a message indicating the absence of the token. The function is also programmed to validate the format of the AuthZ header and extract the token respectively.

Once the token has been retrieved, the function proceeds to authenticate it using the secret key stored in the **SECRET_TOKEN** environment variable. Successful validation allows the user’s information to be decoded from the token. Subsequently, the function queries the user service to acquire the user details from the database, utilizing the extracted user ID. If the user is located and holds the “Manager” role, the function triggers the **next()** method to transfer control to the subsequent middleware or route handler. In contrast, if the user’s role is not “Manager”, the function returns a 401 Unauthorized status accompanied by an error message confirming the user’s lack of AuthZ to access the requested resource.

It is important to note that while the **checkRoleManager** function has been extensively described here, similar functions such as **checkRoleAdmin** and **checkRoleAdminOrManager** adhere to a comparable structure and logic.

The primary distinction lies in the role they seek to validate, ensuring that only users with the pertinent roles can access specific routes and upholding the security and reliability of the application.

```
      const checkRoleManager = async (req, res, next) => {
          let token;
          let authHeader = req.headers['authorization'];

          if (!authHeader)
              return res.status(401).json({ message: 'Token not provided'
});

          if (authHeader.indexOf(' ') >= 0)
              token = authHeader.split(' ')[1];
          else
              token = authHeader
          try{
              const jwtDecoded = jwt.verify(token,
process.env.SECRET_TOKEN);
              const foundManager = await
userService.listManagerById(jwtDecoded.user[0].ID);

              if (foundManager[0].role === utils.userRoles.Manager) {
                  next();
              }
              else {
                  return res.status(401).json({
                      error: 'You are not authorised to view this
information/page'
                  });
              }
          }
          catch (error){
              return res.status(401).json({ error: `${error.message}` });
          }
      }
```

Listing 7: const checkRoleManager

5.1.2.3. _authRoutes.js

The **_authRoutes.js** file, shown in Listing 8, is responsible for defining the routes related to authentication actions in the application, such as login and register. It uses Express.js to create a router that maps HTTP POST requests to the corresponding controller functions responsible for handling these actions.

It starts by importing the necessary modules. Import the express module and create a new router instance using **express.Router()**. In addition, it imports the authentication controller, which contains the logic for handling authentication actions.

The authentication controller is then de-structured to extract the login and register functions. These functions are then mapped to the respective routes using the router. Finally, the routes are exported as an object with the routes property, making them available for use in other parts of the application. This configuration allows the application to handle user login and register via well-defined API endpoints, ensuring that the authentication process is simplified and maintainable.

```
'use strict'
const express = require('express');
const router = express.Router();
const authController = require('../controllers/_authController');

const { login, register } = authController;

router.post('/login', login);
router.post('/register', register);

module.exports = {
    routes: router
};
```

Listing 8: File _authRoutes.js

5.1.2.4. _authController.js

The **_authController.js** file is responsible for configuring routes to manage authentication-related actions within the application. It imports the Express module to create a router and the authentication controller to handle login and register logic. The router maps HTTP POST requests to the /login and /register endpoints to the corresponding login and registration functions in the authentication controller. This file organises the routing of authentication processes, ensuring that login and registration requests are correctly directed to the respective handlers. This structure facilitates a well-organised and easily maintained authentication system within the application.

The **login** function in Listing 9 handles user login by checking the credentials and creating a JWT if the credentials are valid. When a login request is made, the function extracts the user's email and password from the body of the request and calls **userService.listUserByEmail** to get the user's details from the database.

If the user is not found or the user list is empty, the function returns a 403 Forbidden response with an error message. It then uses **bcrypt.compare** to compare the supplied password with the hashed password stored in the database. If the passwords don't match, the function stops. The function checks if the user's account is active. If the account is inactive, it returns a 403 Forbidden response. If the login credentials are correct and the account is active, the function removes the password field from the user object for security reasons and generates a JWT token that expires in 1 hour.

The function creates a cookie with the token and returns a 200 OK response with the authorization token included. This token is used to authenticate subsequent requests for secure communication between the client and the server.

```
    const login = async (req, res, next) => {
        try {
            const email = req.body.email;
            const password = req.body.password;
            const user = await userService.listUserByEmail(email);

            if (!user || !user.length) {
                return res.status(403).json({
                    error: "Invalid Login!"
                });
            }
```

```
            bcrypt.compare(password, user[0].password).then(function
(res) {
                if (res) {
                    true;
                }
            });



            if (user[0].status !== utils.userStatus.Active) {
                return res.status(403).json({
                    error: "This user account is currently Inactive,
please try again later!"
                });
            }


            delete user[0].password;
            const token = jwt.sign({user}, process.env.SECRET_TOKEN, {
expiresIn: "1h" });


            res.cookie("token", token, {
                httpOnly: false,
            })


            return res.status(200).json({Authorization: `${token}`});
        }
        catch (error) {
            res.status(400).send(error.message);
        }
    }
```

Listing 9: const login

The **register** function in Listing 10 manages user registration by creating a new user account, capturing the user's details and generating a JWT upon successful registration. On receiving a registration request, the function extracts the user's email, password and confirm the password from the body of the request.

The function checks if the email provided exists in the database. If it does, it returns a 409 error response with an error message "Email already in use!". If the passwords provided in the password and confirm password fields don't match, it returns a 409 error response with an error message indicating incompatibility.

When the email is not in use and the passwords match, the function hashes the password using **bcrypt.hash**. The hashed password is then assigned back to the body of the request and a new user is created in the database. After creating the user, the function fetches the user's details, generates a JWT token with an expiry of 1 hour, sets a cookie with the token and returns a 200 OK response with the token for secure registration and communication.

```
      const register = async (req, res) => {
          try {
              const email = req.body.email;
              const password = req.body.password;
              const confirmPassword = req.body.confirmPassword;
              const userCheckExist = await
userService.listUserByEmail(email);

              if (Object.keys(userCheckExist).length > 0) {
                  if (userCheckExist?.email == email) {
                      return res.status(409).json({
                          error: "Email already in use!",
                      });
                  }
                  if (userCheckExist[0].password) {
                      delete userCheckExist[0].password;
                  }
              }

              if (password !== confirmPassword) {
                  return res.status(409).json({
                      error: "Password field doesn't match Confirm
Password!"
                  });
              }
```

```
            bcrypt.hash(password, saltRounds).then(function (hash) {
                req.body.password = hash;
            });
            const newUserData = req.body;
            await userService.createNewUser(newUserData);
            const user = await userService.listUserByEmail(email);

            const token = jwt.sign({user}, process.env.SECRET_TOKEN, {
expiresIn: "1h" });

            res.cookie("token", token, {
                httpOnly: false,
            })

            return res.status(200).json({Authorization: `${token}`});
        }
        catch (error) {
            res.status(400).send(error.message);
        }
    };
```

Listing 10: const register

5.1.3. Models

The **models** folder contains files that define the structure and relationships of entities within the application. The companyModel.js file outlines the Company model, representing the companies in the application, while the paymentModel.js file outlines the Payment model, which means the payments made in the application. These models are established using Sequelize, a well-known ORM library for Node.js.

Each model corresponds to a table in the database and specifies the entity's attributes, their respective data types and their relationships with other entities. These models form the application's data structure base, facilitating efficient data management and relational mapping between various entities.

5.1.4. Routes

The route files located in the **routes** directory serve to define endpoints for various entities within the application, linking them to the corresponding controller functions by using Express.js to handle HTTP requests. The **_authRoutes.js** file oversees authentication actions, such as logging in and registering, while the **serviceUserRoutes.js** file manages the administration of the service's users, and so on. These route files ensure that the application's endpoints are well-defined, secure and correctly linked to the respective controller functions, thereby facilitating the efficient handling of requests and data management.

5.1.5. Controllers

The controllers located in the **controllers** directory are responsible for managing HTTP requests and communicating with services to perform CRUD operations. The **_authController.js** is responsible for handling authentication, the **companyController.js** oversees actions related to the company, the **paymentController.js** manages payment operations, and so on. These controllers guarantee the correct execution of business logic to ensure efficient order processing and data management.

5.1.6. Services

The **service** directory includes files that manage the business logic and database interactions for various entities within the application. For example, the **companyService.js** file oversees company-related operations, the **paymentService.js** file handles payment-related operations and the **userService.js** file manages user-related operations.

Each file contains functions for retrieving, creating, updating and deleting database records relating to its specific entity. Collectively, these files ensure that the application's business logic is properly managed, facilitating efficient data management and interactions with the database through well-defined functions for each entity.

5.2. Front-end

The project was developed with a modern, responsive front-end using Angular.js to ensure a dynamic and efficient user experience. The application is divided into three main front-ends, each designed to interact seamlessly with the centralised authentication system. The fundamental components of the front-ends were supplied by the ARI department.

To maintain secure and consistent user authentication throughout the application, a centralised authentication system was implemented. This system handles user login, registration and session management, providing a unified authentication experience for all front-ends. The authentication service communicates with a back-end API to verify credentials and manage user sessions.

5.2.1. D2C

The D2C platform is designed to be responsive and user-friendly, serving as the main interface for e-commerce platform managers. It facilitates the easy integration of various services, including recommendation engines and virtual fitting rooms, into online shops. The interface offers user-friendly controls and comprehensive management tools to improve the operational efficiency and quality of service of e-commerce platforms.

5.2.1.1. Register

Figure 17Figure 1 shows the elegant and user-friendly registration page for the "D2C Portal" application. The simple layout features a registration form in the centre with fields for email, password and password confirmation. A prominent blue "Registar" button and a "Já tem uma conta? Login" link are also visible. This design encourages users to easily create new accounts.

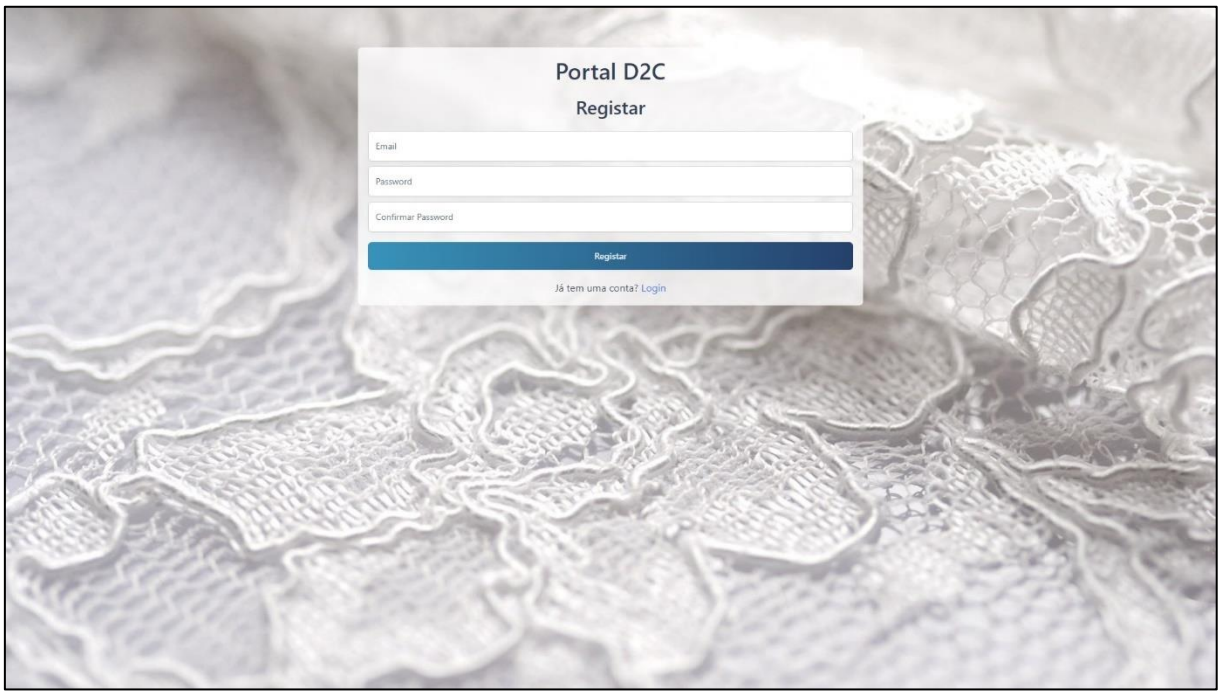


Figure 17: D2C Register

5.2.1.2. Login

The login page for the **Portal D2C** application is shown in Figure 18. It features an elegant, minimalist user interface with a login form positioned in the centre. The form includes fields for entering an email address and password, as well as a blue "Login" button for submission.

Below the login form is a link labelled “Não tem uma conta? Registar” which takes users to the registration page. The design aims to provide a user-friendly and visually appealing experience, ensuring smooth access.

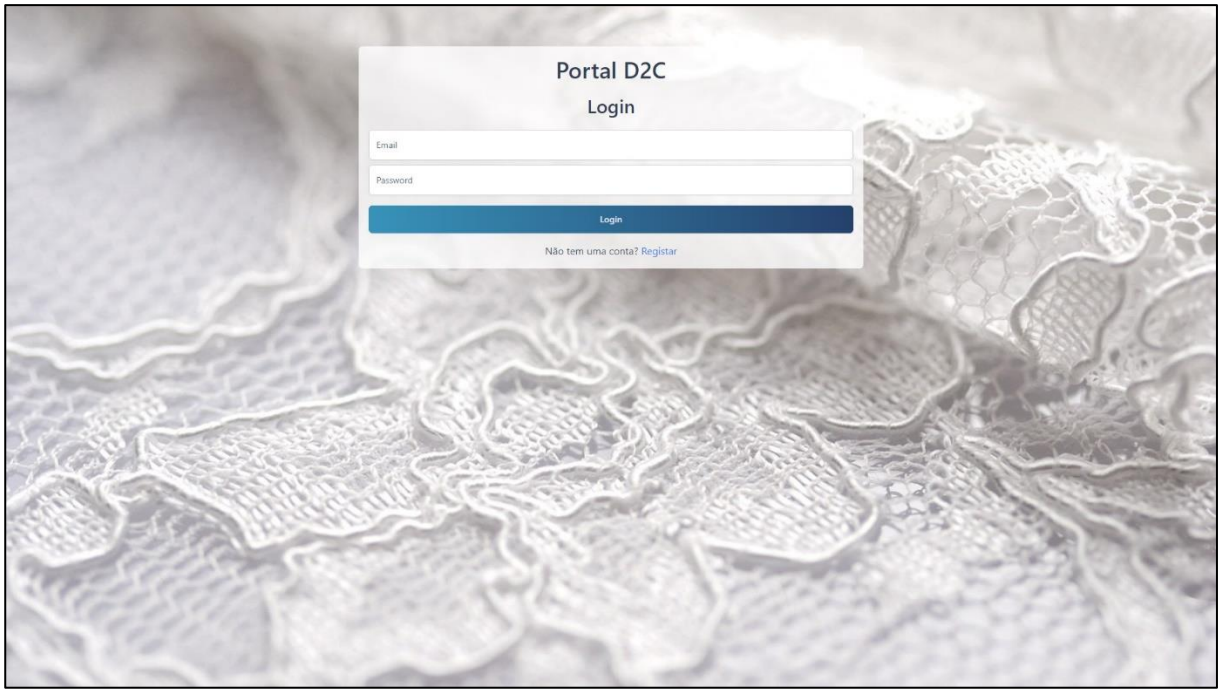


Figure 18: D2C Login

5.2.1.3. Home

In Figure 19, the main dashboard of the **Portal D2C** application is displayed after the user has logged in. The interface welcomes the user with a friendly message and encourages them to explore and organise the available services according to their needs. The dashboard prominently displays the "Serviços Disponíveis" section, which shows various services offered by the platform. Each service card includes the service name, type, description, starting price and a Details button for more information.

The top navigation bar provides links to different sections, such as "Início", "Serviços Disponíveis", "Os meus serviços" and an option to log out with "Terminar Sessão". The design is intuitive and user-friendly, making it easy to navigate and access the portal's services.

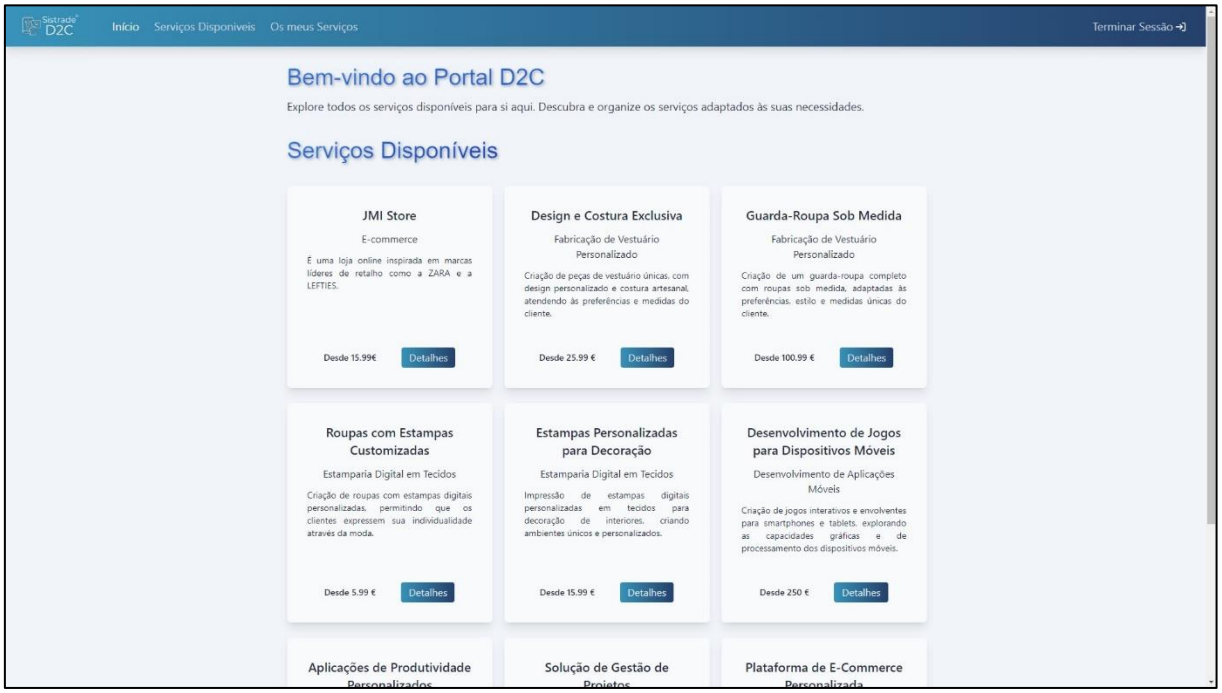


Figure 19: D2C Home

5.2.1.4. Services Available

In Figure 20, you can see the Available Services page of the **D2C Portal** application. This page presents all the available services in a card format. Each card contains the service name, type, description, starting price, and a Details button for additional information. The top navigation bar allows users to switch between various portal sections, including "Início", "Serviços Disponiveis", and "Os meus Serviços".

Users can also log out by selecting the "Terminar Sessão" option. The design focuses on usability and accessibility, displaying services in an organized and easily navigable manner to help users quickly find and choose the services that meet their needs.

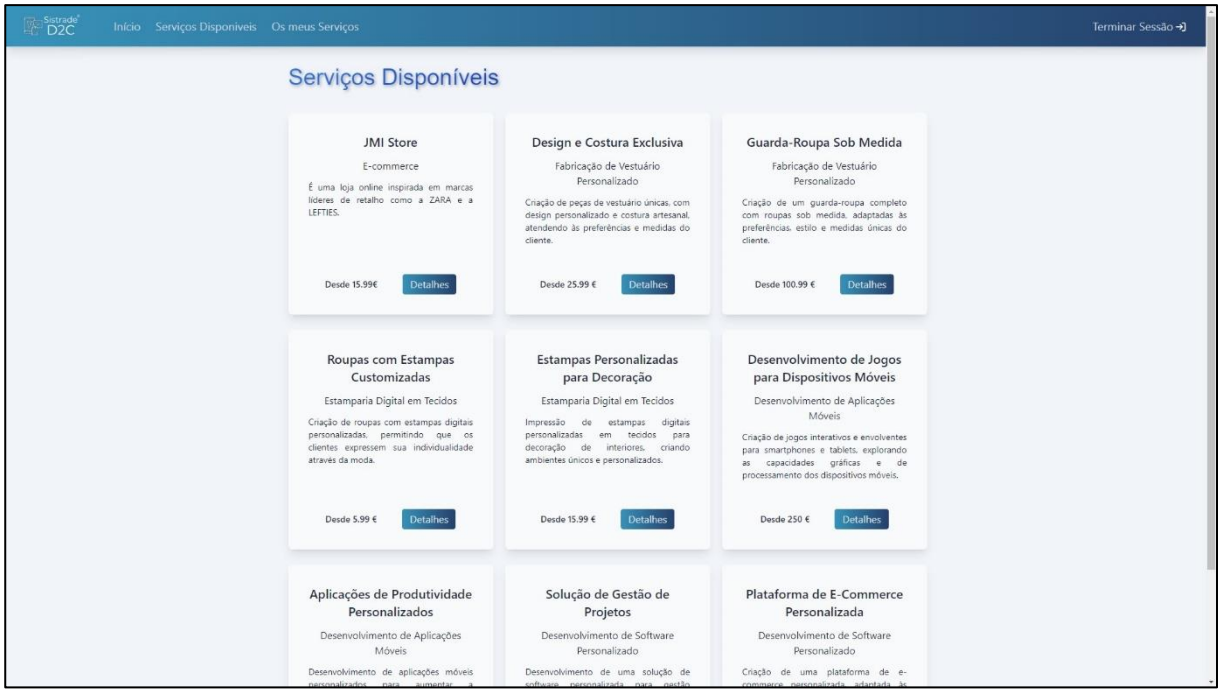


Figure 20: D2C Services Available

5.2.1.5. My Services

In Figure 21, the My Services page of the **Portal D2C** application is shown. This page shows the services that the user has selected or subscribed to. Currently, there is only one service card for the **JMI Store**, which is an e-commerce service.

The card contains the name of the service, the type, a brief description and the starting price. In addition, there is a Details button to access more information. The top navigation bar allows the user to switch between the different sections of the portal, such as "Início", "Serviços Disponíveis" and "Os meus Serviços".

It also provides an option to end the session, called "Terminar Sessão". The design is clean and easy to use, allowing users to easily visualise and manage their subscribed services.

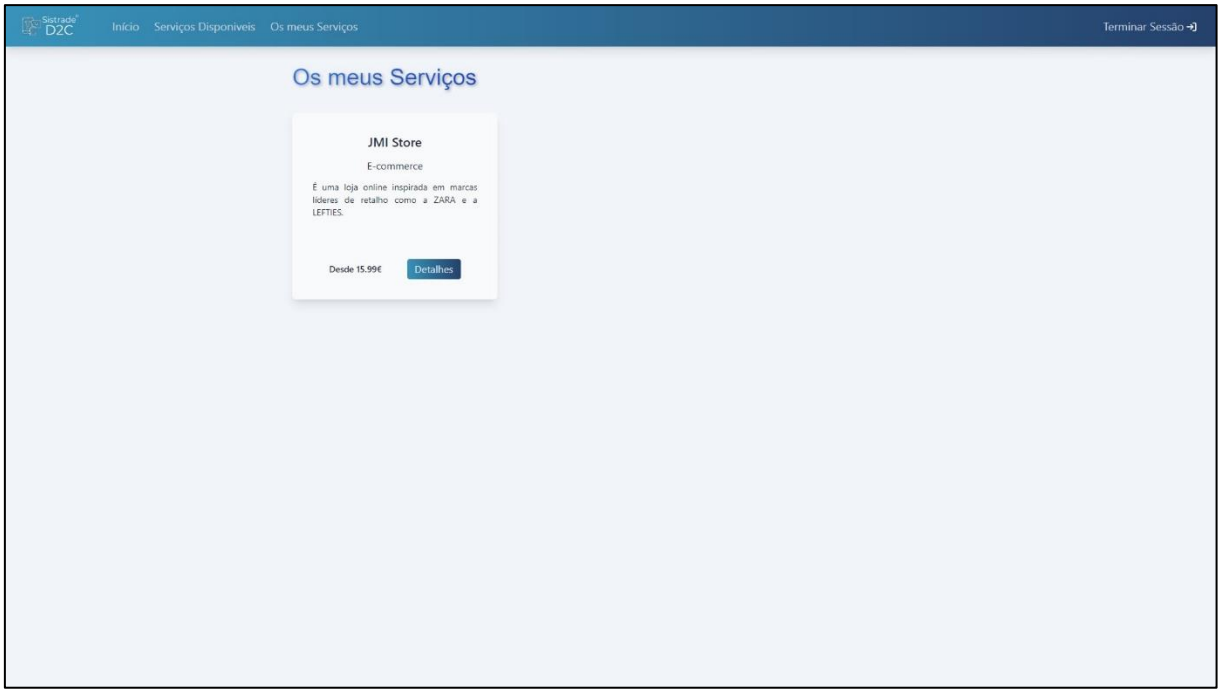


Figure 21: D2C My Services

5.2.2. E-commerce JMI

The E-commerce JMI store effectively simulates the customer experience on an e-commerce platform. It demonstrates how customers can authenticate once and access a variety of integrated services. This simulation shows the process of seamless AuthN and highlights the benefits of a centralized login system that increases customer convenience and engagement.

5.2.2.1. Register

As shown in Figure 22, the registration page for the **E-commerce JMI** application. The user interface features a central registration form on a grey folded fabric background, which gives it an elegant and professional look. The form consists of fields for entering an e-mail address, a password and confirming the password. Below these fields is a blue "Registar" button for submitting the registration form.

In addition, there is a link that allows users to switch to the login page if they already have an account, labelled "Já tem uma conta? Login". The design is intended to offer a user-friendly and visually appealing experience, encouraging new users to sign up for the e-commerce platform.

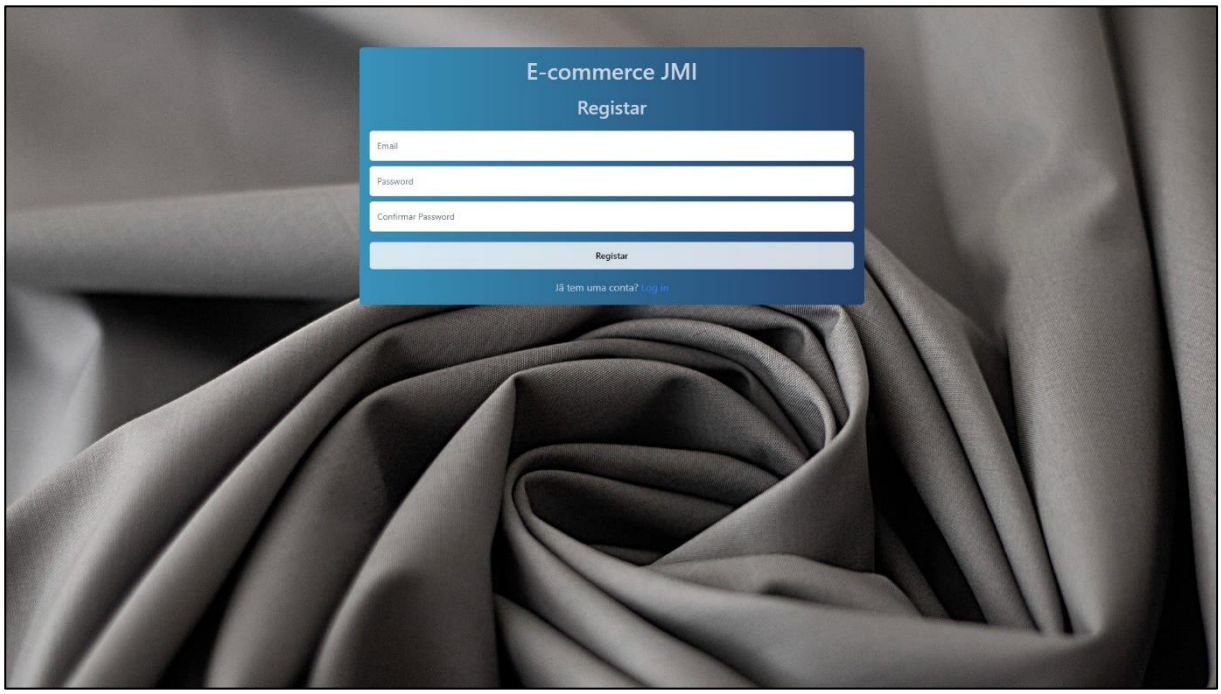


Figure 22: JMI Store Register

5.2.2.2. Login

In Figure 23, the login page of the **E-commerce JMI** application is shown. The user interface features a login form positioned centrally against a backdrop of folded grey fabric, giving it a clean and professional look. The form includes fields for entering an e-mail address and password, with a blue "Login" button positioned underneath for submitting the login information. In addition, there is a link for users without an account to navigate to the registration page, labelled "Não tem uma conta? Registar".

The design remains consistent with the registration page, to provide a user-friendly and visually appealing experience, making it easier for users to log in to the e-commerce platform.

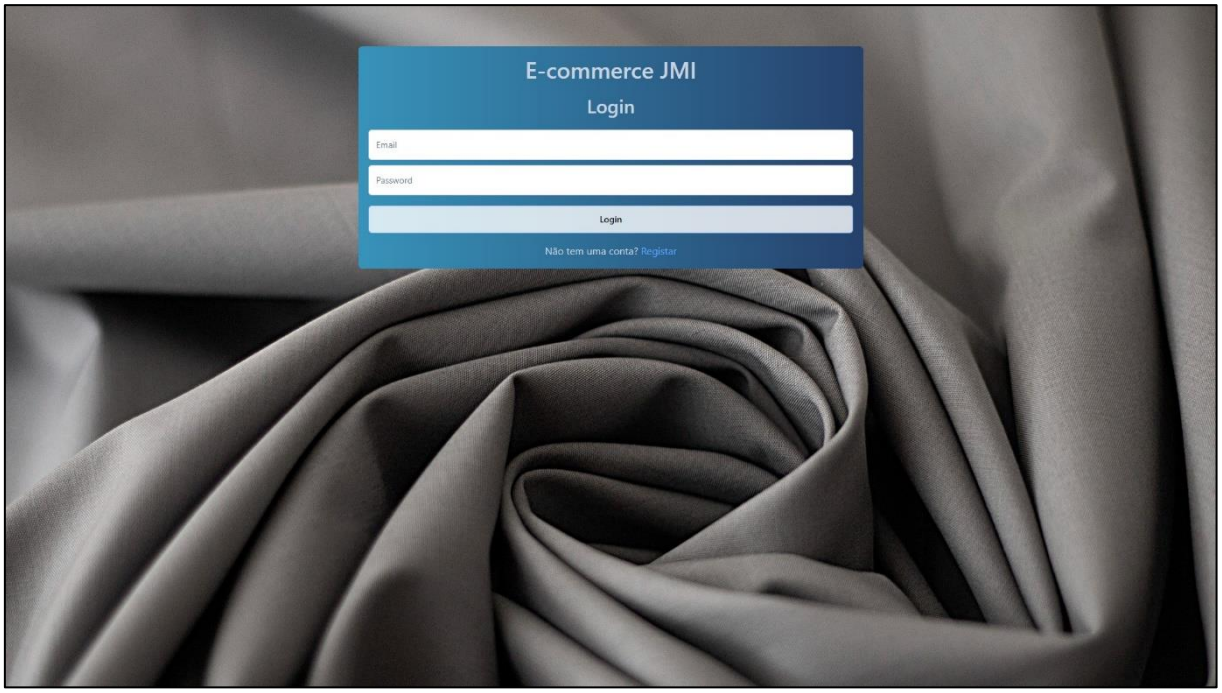


Figure 23: JMI Store Login

5.2.2.3. Home

After successfully logging in, users are greeted with the **E-commerce JMI** landing page, as shown in Figure 24. The page confirms the successful login and presents the platform as an e-commerce site inspired by popular retail brands such as ZARA and LEFTIES. Users are encouraged to explore the platform's features and enjoy a smooth shopping experience.

The top navigation bar features an "Início" link and a "Terminar Sessão" option, along with a "Service de Meteorologia" button that directs users to the **Weather Service** that is available on the platform. The design is minimalist and user-friendly, guiding users to easily navigate the e-commerce platform.

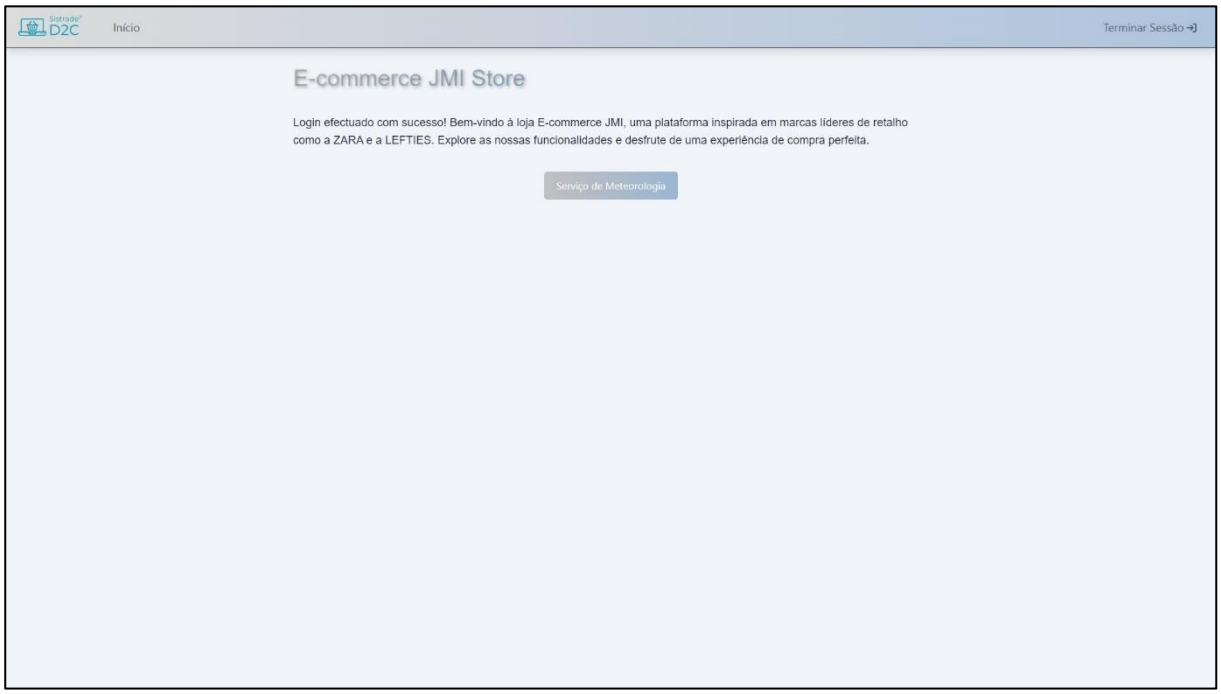


Figure 24: JMI Store Home

5.2.3. Weather Service

Weather Service simulates an external service, such as a weather forecast in the city of Porto, to demonstrate the versatility and convenience of the centralized AuthN system. It shows how users can access external services with a single login, eliminating the need for multiple AuthNs. This simulation emphasises the improved user experience by providing easy and consistent additional services, reducing complexity and improving usability.

5.2.3.1. Register

In Figure 25, the registration page for the **Tempo JMI** application is shown. The user interface features a central registration form with a background of soft clouds, creating an atmosphere of peace and calm. The form includes fields for entering an email address, a password and password confirmation. Below these fields, there is a visible red "Registar" button for submitting the registration form.

In addition, there is a link that allows users to switch to the login page if they already have an account, labelled "Já tem uma conta? Login". The design aims to provide a user-friendly and visually appealing experience to encourage new users.

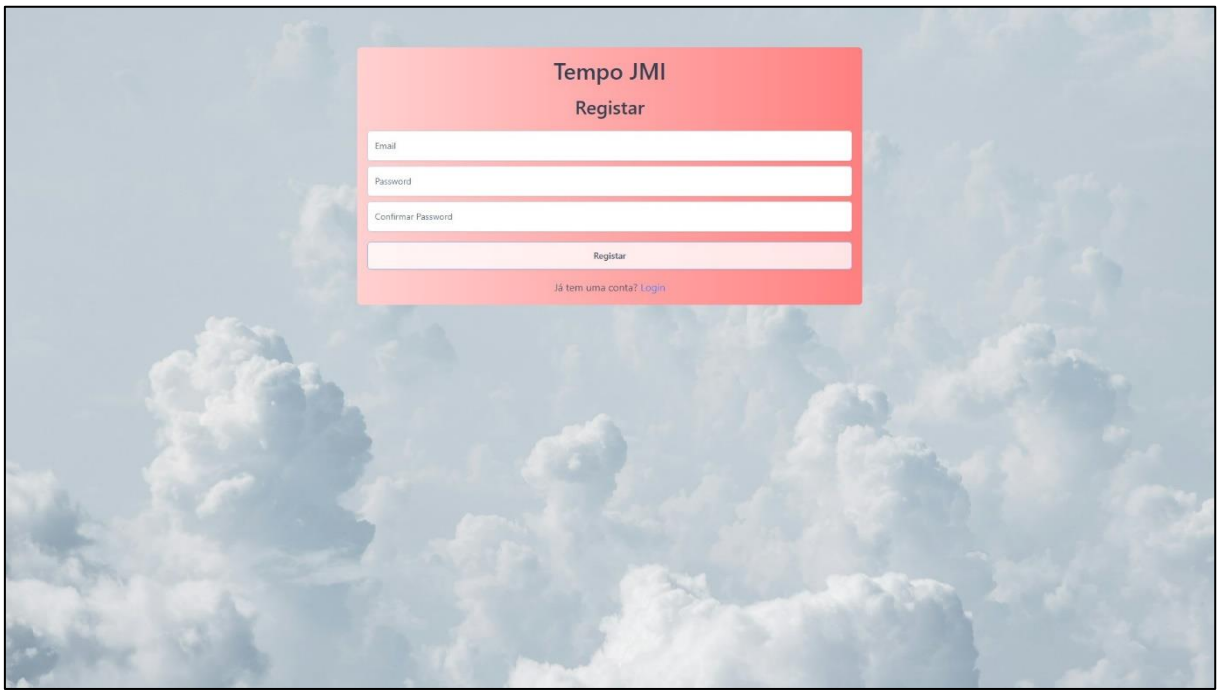


Figure 25: Weather Service Register

5.2.3.2. Login

Below in Figure 26 is the login page for the **Tempo JMI** application. The user interface features a central login form with the same background as the register page. The form consists of fields for entering an e-mail address and password, along with a prominent red "Login" button for submitting login details. In addition, there is a link for users without an account to navigate to the registration page, labelled " Não tem uma conta? Registar ".

The design is in line with the registration page, with the intention of providing a user-friendly and aesthetically pleasing experience, simplifying the login process for users accessing the meteorological services offered by the **Tempo JMI** platform.



Figure 26: Weather Service Login

5.2.3.3. Home

Figure 27 shows the destination page of the **Tempo JMI** application after a successful login. The page welcomes the user with a message confirming the successful login and presents a concise overview of the application. It highlights the fact that **Tempo JMI** shows the weather in Porto.

The top navigation bar includes a link called "Início" to the home page and an option called "Terminar Sessão" to end the session. In the centre, there is a button labelled "Tempo Hoje em Porto", which encourages users to check the current weather in Porto. The design is characterised by its simplicity and minimalism, with a focus on presenting crucial information and a clear call to action to check the weather.



Figure 27: Weather Service Home

5.3. Tests of the Solution

To confirm the effectiveness of our application's authentication and authorization mechanisms, we performed a comprehensive set of tests utilizing Postman for API testing and acceptance tests for overall system validation.

5.3.1. Postman

The tests on Postman were carried out to validate the registration, login and user authorization processes. The registration process required initiating a POST request to the /register endpoint with accurate data and waiting for a 201 (Created) status in response. For the login process, a POST request to the /login endpoint with the correct credentials was anticipated to return a 200 (OK) status along with a JWT token. Authorised access required a GET request to a secure endpoint with a JWT token, resulting in a 200 (OK) status.

Unauthorised access tests included sending GET requests to a secure endpoint without a token or with an invalid token, with an expected response of 401 (Unauthorised). Role-based authorization tests involved sending GET requests to endpoints that required specific permissions to validate access according to the user's role.

5.3.1.1. Register

In Figure 28, the Postman interface is used to test the registration endpoint of an API. The request configuration at the top specifies the use of the HTTP POST method and the specified URL.

The header section often contains details such as Content-Type and Authorization. The body of the request is defined as raw and in JSON format, containing user registration details such as name, e-mail address, password and password confirmation.

At the bottom of the interface, the server response shows a status code of 200 OK, which means that the request has been completed. The response time is shown with the response size. The body of the response contains an authorization token in the form of a JWT, which can be used to authenticate future requests to secure endpoints.

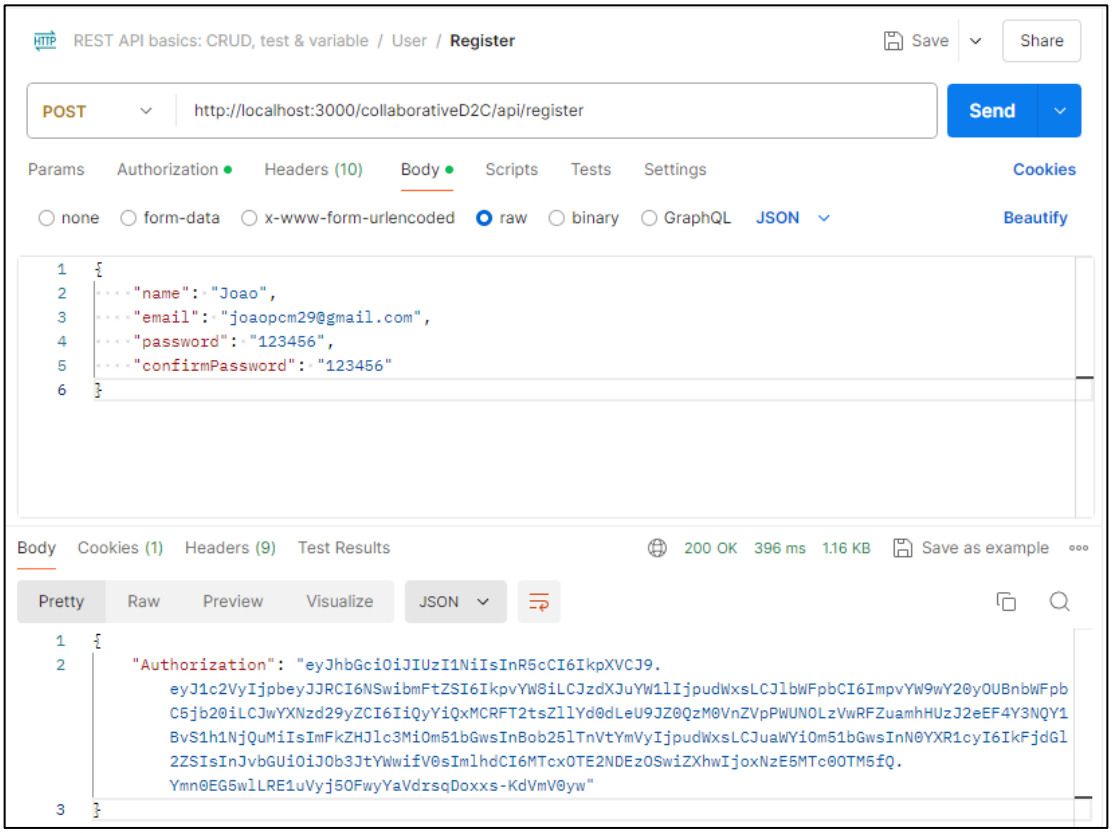


Figure 28: Register Tests

5.3.1.2. Login

In Figure 29, the Postman interface is shown testing the login endpoint of an API. The HTTP method is set to POST, indicating that data is being sent to the server.

The headers section reveals that there are 11 headers configured for the request. The body of the request is set to raw and in JSON format, containing the user's login details, specifically the user's email and password.

The server responds with a 200 OK status code, indicating a successful request. The response includes an authorization token to authenticate future requests to secure endpoints.

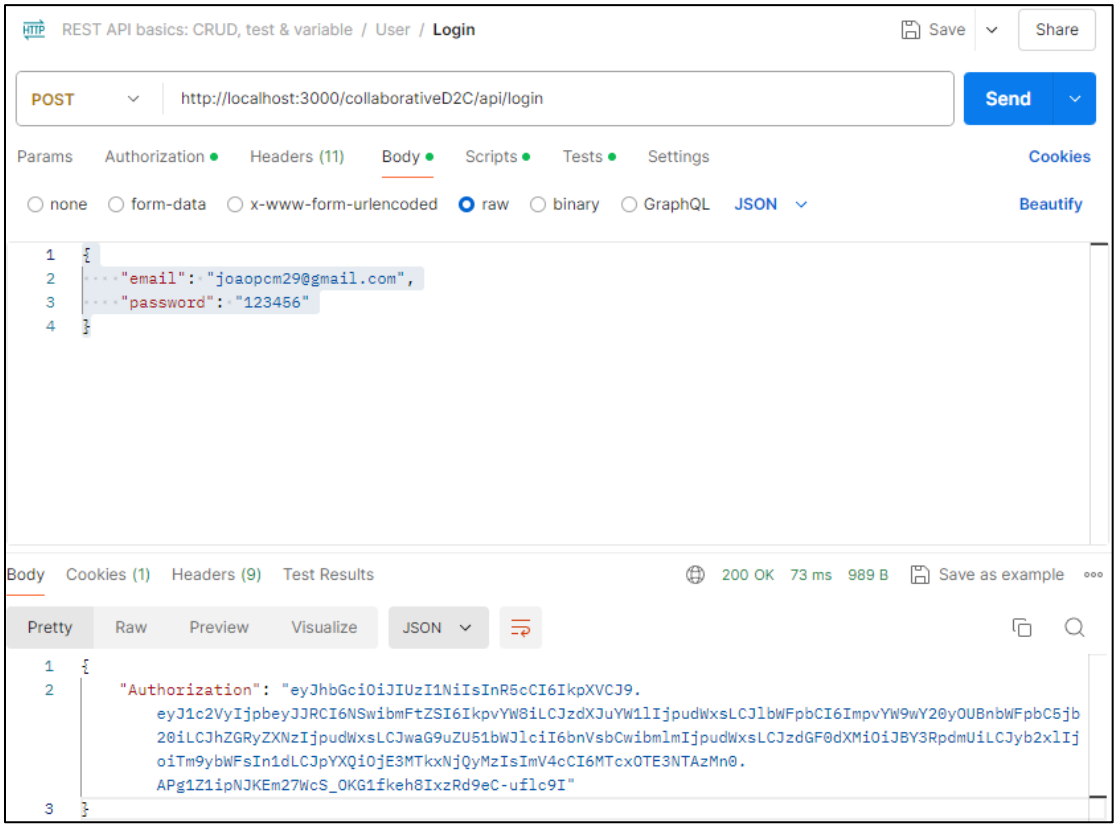


Figure 29: Login Tests

5.3.2. Acceptance Tests

The acceptance tests were designed to validate the entire system as shown below in Figure 30, ensuring that all the components and their interactions work correctly in an integrated environment. The validation process involved exhaustive testing of the authentication and authorization mechanisms in different services of the **Portal D2C** application. The diagram shown illustrates the main interactions between the e-commerce service and the meteorological service, both of which depend on the central **Portal D2C** for authentication.

The validation process focused on confirming users' ability to successfully log in and authenticate on the E-commerce platform. This included validating centralized authentication, ensuring seamless transfer of authentication tokens between different services, and verifying that protected resources are accessible only to authenticated and authorized users.

The process of the user logging in to the e-commerce service involves several steps. Initially, the user enters their login credentials on the e-commerce login page. Subsequently, an authentication request is sent to the **Portal D2C**. The **Portal D2C** then generates a token and returns it to the e-commerce service. The e-commerce service uses this token to authenticate and validate the user. Once validated, the e-commerce service grants access to the protected resources, which are then returned to the user.

Navigating to the weather service also follows a specific process. After accessing the e-commerce service, the user moves from the JMI shop to the meteorological service. The Weather service requests authentication from the **Portal D2C**. The **Portal D2C** checks that the user is already logged in and recognises the existing session. The Meteorology service then uses the token previously received to re-authenticate the user. The token is validated by the **Portal D2C** and, after successful validation, the meteorological service grants access to its protected resources and returns them to the user.

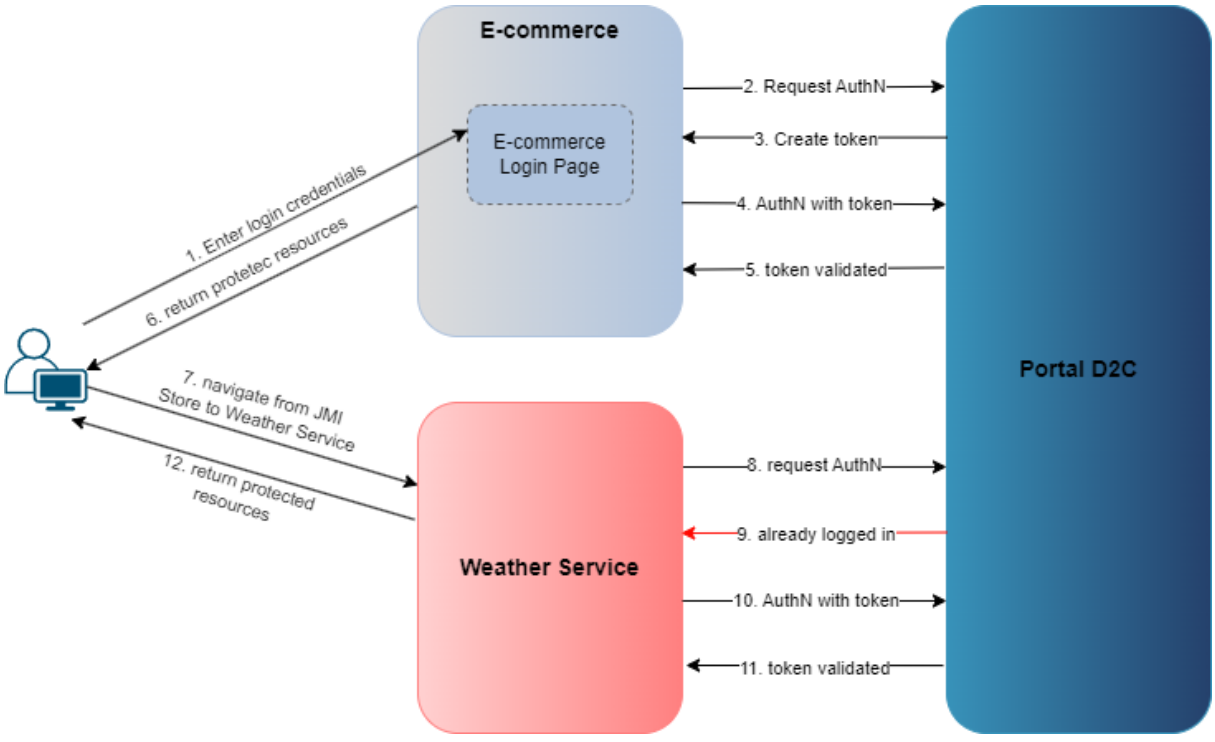


Figure 30: AuthN and AuthZ workflow in the D2C Portal system

6. Conclusion

The project aimed to develop a centralized authentication module to simplify user authentication. It aimed to address the issue of repetitive authentication in various services and systems by providing a secure and effective solution.

The project successfully created a centralized authentication module, allowing users to log in once and seamlessly access various services. Security was improved by implementing JWT tokens for identity validation and cookies for consistent authentication across multiple sites. The user experience was also enhanced by simplifying access to various services through a unified authentication mechanism.

The project was organised around multiple specific objectives to develop and implement solutions, carefully planned and executed to respond to the needs identified and improve the user experience.

O1: The Centralised Authentication Module has been developed, based on JWT tokens, making the authentication process simpler for users.

O2: A responsive and user-friendly front-end for the D2C platform has been created using Angular.js, enhancing the user experience.

O3: A front-end application with authentication has been developed to simulate the use of an e-commerce platform.

O4: Another front-end application with authentication has been developed to simulate an external service, such as a weather forecast.

O5: The authentication module has been successfully integrated into the systems described in objectives **O2**, **O3**, and **O4**.

O6: The authentication module has been thoroughly validated as part of a complete system, demonstrating its ability to provide automated, continuous, and secure access to various resources.

Bibliography

Aarness, A. (11 de January de 2024). *ENDPOINT SECURITY: HOW ENDPOINT PROTECTION WORKS*. Obtido em 26 de Feb de 2024, de CrowdStrike: https://www.crowdstrike.com/cybersecurity-101/endpoint-security/

Check Point Software Technologies. (2024). *The Different Types of Cybersecurity*. Obtido em 26 de Feb de 2024, de Check Point: https://www.checkpoint.com/cyber-hub/cyber-security/what-is-cybersecurity/

Cloudflare, I. (2024). *What is SSO? | How single sign-on works*. Obtido em 06 de Mar de 2024, de Cloudflare: https://www.cloudflare.com/learning/access-management/what-is-sso/

DataFlair. (2024). *AngularJs Advantages and Disadvantages – Why AngularJS is Popular*. Obtido em 29 de Feb de 2024, de DataFlair: https://data-flair.training/blogs/angularjs-advantages-and-disadvantages/

Dennis, A. &. (2009). *Systems Analysis & Design.* John Wiley & Sons, Inc. Obtido em 11 de Mar de 2024

Dhaduk, H. (02 de February de 2024). *Angular vs React 2024: Which Framework to Choose for Your Project?* Obtido em 29 de Feb de 2024, de Simform: https://www.simform.com/blog/angular-vs-react/

EPAM Systems Inc. (24 de February de 2024). *Top 5 NodeJS pros and cons: what they mean for your project*. Obtido em 04 de Mar de 2024, de Epam Anywhere: https://anywhere.epam.com/en/blog/node-js-pros-and-cons

Frontegg. (24 de February de 2022). *Authentication: Methods, Protocols, and Strategies*. Obtido em 27 de Feb de 2024, de Frontegg: https://frontegg.com/blog/authentication

IBM. (2024). *What is cloud security?* Obtido em 29 de Abr de 2024, de IBM: https://www.ibm.com/topics/cloud-security

IBM. (2024). *What is network security?* Obtido em 29 de Abr de 2024, de IBM: https://www.ibm.com/topics/network-security

Identity, O. (2024). *Authentication vs. Authorization*. Obtido em 27 de Feb de 2024, de OneLogin: https://www.onelogin.com/learn/authentication-vs-authorization

ISO/IEC. (25 de October de 2022). *ISO/IEC 27001:2022*. Obtido em 07 de Mar de 2024, de ISO.org: https://www.iso.org/standard/27001

Maratov, A. (08 de August de 2017). *Central Authentication Service (CAS) implementation using Django microservices*. Obtido de Medium: https://medium.com/@adiletmaratov/central-authentication-service-cas-implementation-using-django-microservices-70c4c50d5b6f

Microsoft. (14 de Feb de 2024). *Microsoft Learn*. Obtido em 05 de Mar de 2024, de Microsoft: https://learn.microsoft.com/en-us/sql/sql-server/what-is-sql-server?view=sql-server-ver16

Okta. (09 de November de 2023). *Central Authentication Service (CAS) Protocol Explained*. Obtido em 28 de Feb de 2024, de Okta: https://www.okta.com/identity-101/central-authentication-service/

Okta, A. b. (27 de November de 2023). *Introduction to JSON Web Tokens*. Obtido em 06 de Mar de 2024, de JWT: https://jwt.io/introduction

Okta, Inc. (2024). *O que é o OpenID Connect (OIDC)?* Obtido em 06 de Mar de 2024, de Auth0 by Okta: https://auth0.com/pt/intro-to-iam/what-is-openid-connect-oidc

One Identity. (2024). *SAML Explained in Plain English*. Obtido em 06 de Mar de 2024, de OneLogin: https://www.onelogin.com/learn/saml

Oracle. (2024). *Database*. Obtido em 05 de Mar de 2024, de Oracle: https://www.oracle.com/database/

Oracle, J. &. (2024). *What is Java and why do I need it?* Obtido em 04 de Mar de 2024, de Java: https://www.java.com/en/download/help/whatis_java.html

Technologies, I. (6 de Aug de 2019). *What are web frameworks and why you need them?* Obtido em 28 de Feb de 2024, de Medium: https://intelegain-technologies.medium.com/what-are-web-frameworks-and-why-you-need-them-c4e8806bd0fb

Unhelkar, B. (2020). *Software Engineering with UML.* CRC Press. Obtido em 12 de Mar de 2024